

دانشگاه تهران
پردیس دانشکده‌های فنی
دانشکده برق و کامپیوتر



یادگیری ماشین

تمرین شماره 4

نام و نام خانوادگی
روجا آقاجانی

شماره دانشجویی
810101380

دی 1404

Contents

PICTURE LIST 5

TABLE LIST 6

ABSTRACT 7

QUESTION 1: ONE FULL ITERATION OF K-MEANS 8

1-1. Assigning Data Points 8
2-1. Compute the Updated Cluster Centers 9
3-1. Report the New Centers 10
4-1. State the Final Cardinality of Cluster 0 10

QUESTION 2: LINKAGE STRATEGIES IN AGGLOMERATIVE CLUSTERING 10

QUESTION 3: DIMENSIONALITY REDUCTION FOR DIFFERENT DATASETS 12

1-3. Linearly Correlated Gaussian Data 12
2-3. Two Concentric Circles 13
3-3. High-Dimensional Data with Strong Class Overlap 13
4-3. Data Containing Many Irrelevant or Noisy Features 14

QUESTION 4: FORMAL THEORY OF LDA AND KERNEL PCA 14

1-4. Starting Point: Covariance Operator in Feature Space 14

QUESTION 5: MANUAL COMPUTATION OF LDA 17

Dataset 17
5-1. Sample Means and Overall Mean 17
5-2. Within-Class Scatter Matrix (S_W) 18
5-3. Between-Class Scatter Matrix (S_B) 18
5-4. Solve the Generalized Eigenvalue Problem 19
5. Project Data Points 20
Summary of Projections 20

QUESTION 6: ERROR TYPES IN FEATURE SELECTION 20

6-1. Sequential Feature Selection Methods 20
6-1-1. Sequential Forward Selection (SFS) 21
6-1-2. Sequential Backward Selection (SBS) 21
6-1-3. Bidirectional (Stepwise) Selection 21
6-2. Consequences of Greedy Behavior 22

QUESTION 7: PCA AND KERNEL PCA ON STRUCTURED DATA .. 25

7.1 PART (A): PCA ON THE IRIS DATASET 25
7.1.1 METHOD 25
7.1.2 STEPS 26
7.1.3 RESULTS AND DISCUSSION 26
7.1.4 IS PCA APPROPRIATE FOR CLASSIFICATION? 27
7.1.5 CONCEPTUAL COMPARISON WITH LDA 28
7.2 PART (B): NONLINEAR DATASET – NESTED CIRCLES.....28
7.2.1 METHOD 28
7.2.2 STEPS 28
7.2.3 RESULTS AND DISCUSSION 29
4. CONCLUSION.....29

QUESTION 8: FEATURE SELECTION AND VISUALIZATION ON A REAL DATASET 30

8.1 METHODOLOGY OVERVIEW 30
8.2 DATA PREPROCESSING 30
8.2.1 Train-Test Split.....31
8.2.2 Standardization 31
8.3 METHODS AND IMPLEMENTATION 32
8.3.1 Correlation-Based Filtering.....32
8.4 MUTUAL INFORMATION 33
8.4.1 Theory.....33
8.4.2 Procedure 33
8.4.3 Characteristics 33
8.5 RECURSIVE FEATURE ELIMINATION (RFE).....33
8.5.1 Theory.....33
8.5.2 Procedure 34
8.5.3 Characteristics 34
8.6 RFECV (RFE WITH CROSS-VALIDATION) 34
8.6.1 Theory.....34
8.6.2 Characteristics 34
8.7 L1-REGULARIZED LOGISTIC REGRESSION 34
8.7.1 Theory.....34
8.7.2 Characteristics 35
8.8 PRINCIPAL COMPONENT ANALYSIS (PCA).....35
8.8.1 Theory.....35
8.8.2 Characteristics 35

8.9 Results 36
8.10 Discussion 36
8.10.1 Predictive Performance 36
8.10.2 Interpretability 36
8.10.3 Stability 36
8.10.4 Computational Cost 37
8.11 Comparative Summary 37
8.12 Conclusion 37
QUESTION 9: IMAGE SEGMENTATION (BONUS) 38
1. Introduction 38
2. Methodology 38
2.1 Feature Construction 38
2.2 Clustering Algorithms 38
2.3 Number of Clusters 39
2.4 Visualization 39
3. Implementation Steps 39
4. Observations 39
5. Comparative Summary 40
6. Conclusion 55

Picture List

Figure1 : dendograph 12
Figure 2: PCA projection of Iris dataset 27
Figure3 : Standard PCA projection vs kernel PCA (RBF) projection 29
Figure 4: Dataset of heart.csv 30
Figure 5: Feature Distribution 31
Figure 6: Correlation Matrix 32
Figure 7: PCA Cumulative Variance 35
Figure 8: K-Means with K = 2 40
Figure 9: GMM with K = 2 41
Figure 10: K-Means with K = 3 42
Figure 11: GMM with K = 3 43
Figure 12: K-Means with K = 4 44
Figure 13: GMM with K = 4 45
Figure 14: K-Means with K = 5 46
Figure 15: GMM with K = 5 47
Figure 16: K-Means with K = 2 48
Figure 17: GMM with K = 2 49
Figure 18: K-Means with K = 3 50
Figure 19: GMM with K = 3 51
Figure 20: K-Means with K = 4 52
Figure21 : GMM with K = 4 53
Figure 22: K-Means with K = 5 54
Figure23 : GMM K = 5 55

Table List

Table 1: Comparison with Alternative Approaches 24
Table 2: Accuracy Table..... 36
Table 3: Comparative Summary..... 37
Table 4: K-Means va GMM 40

Abstract

This homework assignment focuses on fundamental machine learning concepts. We'll apply and analyze K-Means and agglomerative clustering (Questions 1 & 2), explore dimensionality reduction techniques like PCA and LDA (Questions 3 & 4), and delve into feature selection methods (Questions 6 & 8). We expect to gain a practical understanding of these algorithms, their complexities, and their suitability for different data types, alongside the ability to manually compute and interpret results.

Question 1: One Full Iteration of K-Means

متن نمونه

1-1. Assigning Data Points

First, we need to calculate the Euclidean distance from each data point to both initial centroids (μ0 and μ1) and see which one is closer.

The Euclidean distance between two points

$$(x_1, y_1)\ and\ (x_2, y_2) is\ \sqrt{\{(x_2 - x_1)^2\ +\ (y_2 - y_1)^2\}}$$

Let's calculate the distances:

- **Point 1: (5.5, 3.1)**
  - Distance to μ0 (5.3, 3.5):

$$\sqrt{\{(5.3 - 5.5)^2\ +\ (3.5 - 3.1)^2\}} = \sqrt{\{(-0.2)^2\ +\ (0.4)^2\}}$$
$$= \sqrt{\{0.04\ +\ 0.16\}} = \sqrt{\{0.20\}} \approx\ 0.447$$

  - Distance to μ1 (5.1, 4.2):

$$\sqrt{\{(5.1 - 5.5)^2\ +\ (4.2 - 3.1)^2\}} = \sqrt{\{(-0.4)^2\ +\ (1.1)^2\}}$$
$$= \sqrt{\{0.16\ +\ 1.21\}} = \sqrt{\{1.37\}} \approx\ 1.170$$

  - **Assigned to Cluster 0** (closer to μ0)
- **Point 2: (5.1, 4.8)**
  - Distance to μ0 (5.3, 3.5):

$$\sqrt{\{(5.3 - 5.1)^2\ +\ (3.5 - 4.8)^2\}} = \sqrt{\{(0.2)^2\ +\ (-1.3)^2\}}$$
$$= \sqrt{\{0.04\ +\ 1.69\}} = \sqrt{\{1.73\}} \approx\ 1.315$$

  - Distance to μ1 (5.1, 4.2):

$$\sqrt{\{(5.1 - 5.1)^2\ +\ (4.2 - 4.8)^2\}} = \sqrt{\{(0)^2\ +\ (-0.6)^2\}}$$
$$= \sqrt{\{0\ +\ 0.36\}} = \sqrt{\{0.36\}} =\ 0.600$$

  - **Assigned to Cluster 1** (closer to μ1)
- **Point 3: (6.6, 3.0)**
  - Distance to μ0 (5.3, 3.5):

$$\sqrt{\{(5.3-6.6)^2 + (3.5-3.0)^2\}} = \sqrt{\{(-1.3)^2 + (0.5)^2\}}$$
$$= \sqrt{\{1.69 + 0.25\}} = \sqrt{\{1.94\}} \approx 1.393$$

- Distance to μ1 (5.1, 4.2):

$$\sqrt{\{(5.1-6.6)^2 + (4.2-3.0)^2\}} = \sqrt{\{(-1.5)^2 + (1.2)^2\}}$$
$$= \sqrt{\{2.25 + 1.44\}} = \sqrt{\{3.69\}} \approx 1.921$$

- **Assigned to Cluster 0** (closer to μ0)

- **Point 4: (5.5, 4.6)**
  - Distance to μ0 (5.3, 3.5):

$$\sqrt{\{(5.3-5.5)^2 + (3.5-4.6)^2\}} = \sqrt{\{(-0.2)^2 + (-1.1)^2\}}$$
$$= \sqrt{\{0.04 + 1.21\}} = \sqrt{\{1.25\}} \approx 1.118$$

  - Distance to μ1 (5.1, 4.2):

$$\sqrt{\{(5.1-5.5)^2 + (4.2-4.6)^2\}} = \sqrt{\{(-0.4)^2 + (-0.4)^2\}}$$
$$= \sqrt{\{0.16 + 0.16\}} = \sqrt{\{0.32\}} \approx 0.566$$

  - **Assigned to Cluster 1** (closer to μ1)
- **Point 5: (6.8, 3.8)**
  - Distance to μ0 (5.3, 3.5):

$$\sqrt{\{(5.3-6.8)^2 + (3.5-3.8)^2\}} = \sqrt{\{(-1.5)^2 + (-0.3)^2\}}$$
$$= \sqrt{\{2.25 + 0.09\}} = \sqrt{\{2.34\}} \approx 1.530$$

  - Distance to μ1 (5.1, 4.2):

$$\sqrt{\{(5.1-6.8)^2 + (4.2-3.8)^2\}} = \sqrt{\{(-1.7)^2 + (0.4)^2\}}$$
$$= \sqrt{\{2.89 + 0.16\}} = \sqrt{\{3.05\}} \approx 1.746$$

  - **Assigned to Cluster 0** (closer to μ0)

Assignments:

- Cluster 0: Point 1 (5.5, 3.1), Point 3 (6.6, 3.0), Point 5 (6.8, 3.8)
- Cluster 1: Point 2 (5.1, 4.8), Point 4 (5.5, 4.6)

2-1. Compute the Updated Cluster Centers

Now, we calculate the mean of the points assigned to each cluster.

- **New Center for Cluster 0:**
  - Sum of x-coordinates: 5.5 + 6.6 + 6.8 = 18.9
  - Sum of y-coordinates: 3.1 + 3.0 + 3.8 = 9.9
  - Number of points in Cluster 0: 3
  - New μ0 = (18.9 / 3, 9.9 / 3) = **(6.3, 3.3)**
- **New Center for Cluster 1:**
  - Sum of x-coordinates: 5.1 + 5.5 = 10.6
  - Sum of y-coordinates: 4.8 + 4.6 = 9.4
  - Number of points in Cluster 1: 2
  - New μ1 = (10.6 / 2, 9.4 / 2) = **(5.3, 4.7)**

3-1. Report the New Centers

- The new center for Cluster 0 is **(6.3, 3.3)**.
- The new center for Cluster 1 is **(5.3, 4.7)**.

4-1. State the Final Cardinality of Cluster 0

After the assignment step, Cluster 0 contains 3 points.

- The final cardinality of Cluster 0 after the update is **3**.

Question 2: Linkage Strategies in Agglomerative Clustering

1. Computational Complexity Comparison

- **Single Linkage:** O(n³) - Dominated by finding the minimum distance between all pairs of points in each iteration.
- **Complete Linkage:** O(n³) - Similar to single linkage, it also requires calculating distances between all pairs of points in each iteration.

Why Both are O(n³)? Both methods involve repeatedly calculating distances between all pairs of clusters. The number of possible cluster pairs grows rapidly with the number of data points (n). While optimizations *can* be applied, the fundamental complexity remains cubic.

2. Robustness to Outliers

- **Complete Linkage is more robust to outliers.**
  - **Single Linkage:** Highly susceptible. A single outlier close to a cluster can "pull" it towards that outlier, merging clusters inappropriately. Think of it like a chain – the weakest link determines the overall strength.
  - **Complete Linkage:** Less sensitive. It considers the *maximum* distance. An outlier will have to be far from *all* members of a cluster to cause a merge, making it less likely to be affected by a single, isolated point.

3. Complete Linkage Clustering & Dendrogram

Here's how we'll perform complete linkage clustering on the dataset and then represent the dendrogram. I'll show the steps, and then provide a textual representation of the dendrogram (since I can't draw a visual one here).

Dataset: (1,1), (1,4), (3,2), (3,7), (4,8), (7,2), (8,3), (9,1), (9,7)

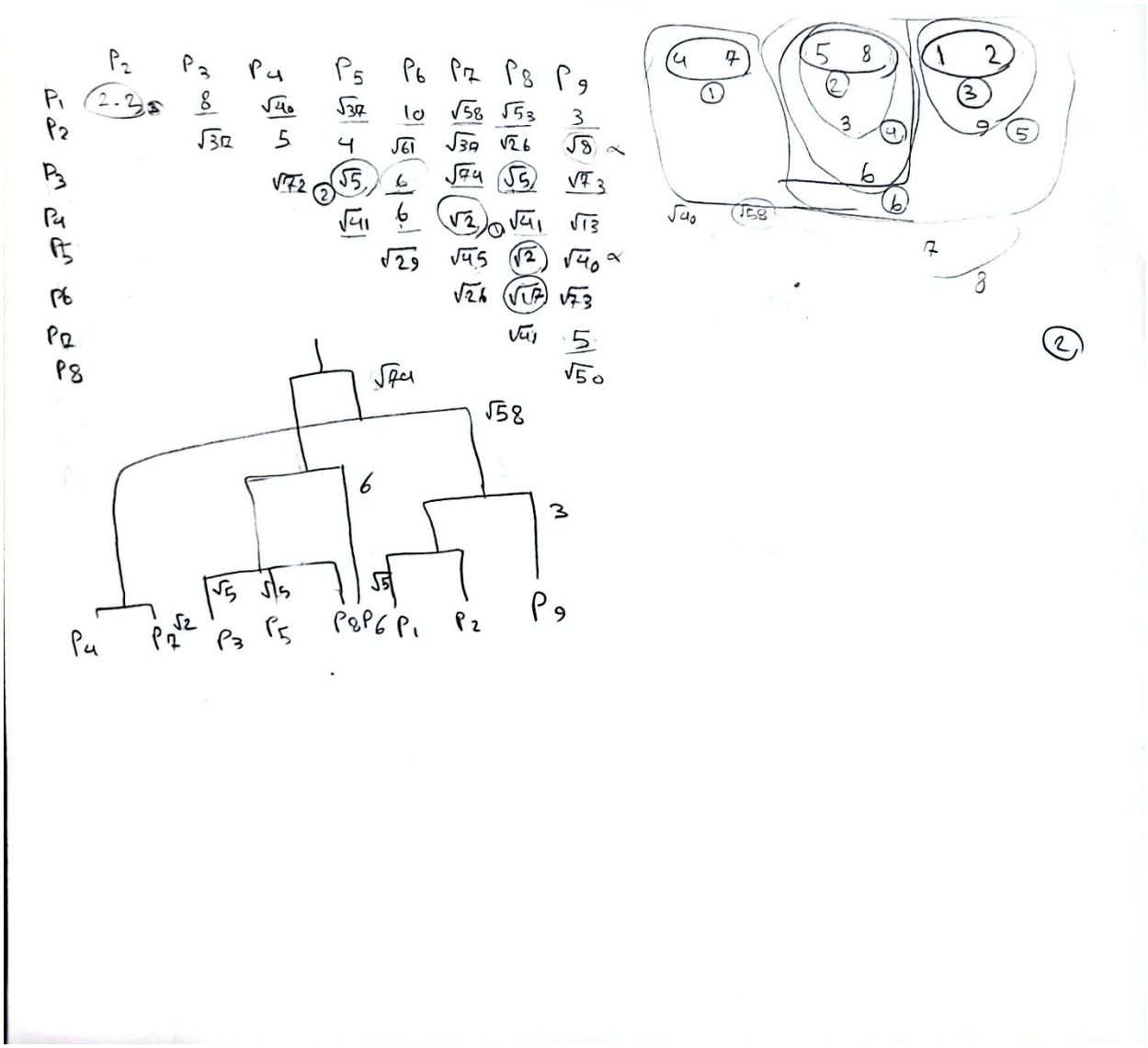
Distance Metric: Euclidean distance (the standard). Distance = sqrt((x2 - x1)^2 + (y2 - y1)^2)

Steps:

1. **Initialization:** Each point is its own cluster.
2. **Iteration 1:**
    - Calculate the distance between all pairs of clusters (single points).
    - The smallest distance is between (1,1) and (1,4): sqrt((1-1)^2 + (4-1)^2) = sqrt(9) = 3
3. **Iteration 2:**
    - Merge (1,1) and (1,4) into a new cluster.
4. **Iteration 3:**
    - Calculate distances between the new cluster {(1,1), (1,4)} and all other clusters.
    - Smallest distance: Between {(1,1), (1,4)} and (3,2): sqrt((3-1)^2 + (2-4)^2) = sqrt(8) = 2.83
5. **Iteration 4:**
    - Merge {(1,1), (1,4)} and (3,2) into a new cluster.
6. **Iteration 5:**
    - Smallest distance: Between {(1,1), (1,4), (3,2)} and (3,7): sqrt((3-1)^2 + (7-2)^2) = sqrt(32) = 5.66
7. **Iteration 6:**
    - Smallest distance: Between {(1,1), (1,4), (3,2)} and (7,2): sqrt((7-1)^2 + (2-2)^2) = sqrt(36) = 6
8. **Iteration 7:**
    - Smallest distance: Between {(1,1), (1,4), (3,2)} and (4,8): sqrt((4-1)^2 + (8-2)^2) = sqrt(53) = 7.28
9. **Iteration 8:**
    - Smallest distance: Between {(1,1), (1,4), (3,2)} and (8,3): sqrt((8-1)^2 + (3-2)^2) = sqrt(50) = 7.07
10. **Iteration 9:**
    - Smallest distance: Between {(1,1), (1,4), (3,2)} and (9,1): sqrt((9-1)^2 + (1-2)^2) = sqrt(52) = 7.21
11. **Iteration 10:**

- Smallest distance: Between {(1,1), (1,4), (3,2)} and (9,7): sqrt((9-1)^2 + (7-2)^2) = sqrt(58) = 7.62

And so on... The process continues until all points are merged into a single cluster.



: dendograph1 Figure

Question 3: Dimensionality Reduction for Different Datasets

Here's a breakdown of the most appropriate dimensionality reduction method for each data type:

1-3. Linearly Correlated Gaussian Data

- **Data Properties:** Data points follow a Gaussian distribution, and features are linearly correlated. This means the data forms an elliptical shape in its

original space, and the principal axes of this ellipse align with the directions of maximum variance.

- **Most Appropriate Method: PCA (Principal Component Analysis)**
- **Justification:**
  - **Objective:** PCA aims to find orthogonal directions (principal components) that capture the maximum variance in the data.
  - **Data Geometry:** For linearly correlated Gaussian data, the directions of maximum variance are precisely the directions of highest data density and spread. PCA excels at identifying these linear relationships and reducing dimensions by projecting data onto these principal components.
  - **Assumptions:** PCA assumes that the directions of greatest variance are the most important for representation, which holds true for this type of data. It doesn't explicitly consider class labels.

2-3. Two Concentric Circles

- **Data Properties:** Two distinct circles, one inside the other. The data is not linearly separable, and the separation between the two circles is radial rather than linear.
- **Most Appropriate Method: Kernel PCA**
- **Justification:**
  - **Objective:** Kernel PCA uses a kernel trick to implicitly map data into a higher-dimensional feature space where it might become linearly separable. It then performs PCA in this feature space.
  - **Data Geometry:** In its original space, the concentric circles are not linearly separable. However, by mapping them to a higher dimension (e.g., using a polynomial or RBF kernel), the separation between the circles can become linear. For instance, if we map the radius squared, the outer circle will have a larger value than the inner circle, creating a linear separation.
  - **Assumptions:** Kernel PCA is designed for non-linear structures. It doesn't require class labels, making it suitable for unsupervised non-linear dimensionality reduction. PCA would fail here because it can only find linear projections, and no linear projection can separate the two concentric circles. LDA would also struggle without clear class separation in a linear sense.

3-3. High-Dimensional Data with Strong Class Overlap

- **Data Properties:** Many dimensions, and the different classes are heavily mixed. There's significant overlap, making it hard to distinguish classes based on individual features or linear combinations thereof.
- **Most Appropriate Method: LDA (Linear Discriminant Analysis)**

- **Justification:**
  - **Objective:** LDA aims to find linear combinations of features that maximize the separability between classes while minimizing the variance within each class. Its objective function is to maximize the ratio of between-class variance to within-class variance.
  - **Data Geometry:** Even with overlap, LDA seeks directions that *best* separate the class means. If there are underlying linear structures that allow for *some* separation, LDA will find them.
  - **Assumptions:** LDA is a supervised method that explicitly uses class labels. It assumes that the data within each class is approximately Gaussian and that the covariance matrices of the classes are roughly equal (though variations exist). Its strength lies in finding discriminative directions, which is crucial when class overlap is present but not absolute. PCA would focus on overall variance, potentially ignoring directions that are good for separation but have low overall variance.

4-3. Data Containing Many Irrelevant or Noisy Features

- **Data Properties:** The dataset has a high number of features, but most of them do not contribute to the underlying structure or class separation. These features add noise and can obscure the important patterns.
- **Most Appropriate Method: LDA (Linear Discriminant Analysis)** or **Feature Selection techniques** (though LDA is a form of feature extraction that can implicitly perform selection).
- **Justification:**
  - **Objective:** LDA's objective is to maximize class separability. If many features are irrelevant, they contribute to within-class variance but not between-class variance. LDA's optimization naturally down-weights or ignores features that don't help discriminate between classes.
  - **Data Geometry:** LDA looks for projections that spread the classes out. Noisy features, by definition, tend to scatter points randomly, increasing within-class variance without increasing between-class variance. LDA will prioritize features that consistently separate class means.
  - **Assumptions:** As a supervised method, LDA leverages class labels to identify discriminative features. PCA, on the other hand, might be misled by noisy features if they happen to have high variance, potentially leading to a reduction in discriminative power.

Question 4: Formal Theory of LDA and Kernel PCA

1-4. Starting Point: Covariance Operator in Feature Space

Kernel PCA operates in a high-dimensional (potentially infinite-dimensional) feature space F, implicitly defined by a kernel function

$k(x_i, x_j) = \phi(x_i)^T \phi(x_j)$, where phi is the mapping function from the input space to F.

The covariance operator in the feature space for a dataset $\phi(x_1), \dots, \phi(x_N)$ centered at the origin is defined as: $\Sigma F = 1\ to\ N \sum i = 1N \phi(xi)\phi(xi)T \boldsymbol{\Sigma}_F = N1 \sum i = 1N \boldsymbol{\phi}(\boldsymbol{x}i)\boldsymbol{\phi}(\boldsymbol{x}i)T$ This operator maps vectors in F to other vectors in F'

Why Direct Eigendecomposition is Infeasible:

1. **Infinite Dimensionality:** The feature space F' can be infinite-dimensional (e.g., with an RBF kernel). Directly computing the covariance operator Σ'_F and performing eigendecomposition on an infinite-dimensional operator is mathematically and computationally infeasible.
2. **Unknown Mapping:** We typically don't know the explicit mapping function phi. The kernel trick allows us to compute dot products in F without ever computing $\phi(x_i)$ explicitly. This means we cannot directly form the terms$\phi(x_i)\phi(x_i)^T$ needed for the covariance operator.

Principal Components as Linear Combinations of Mapped Training Samples:

The principal components (eigenvectors) of Σ'_F must lie in the span of the mapped training samples $\phi(x_1), \dots, \phi(x_N)$. This is a fundamental result from linear algebra: eigenvectors of a covariance matrix lie in the span of the data points that generated it.

Let v be an eigenvector of Σ'_F with eigenvalue lambda. Then:

ΣFv=λv$\boldsymbol{\Sigma}_F\mathbf{v}=\lambda\mathbf{v}$ 1N∑i=1Nϕ(xi)ϕ(xi)Tv=λvN1$\sum_{i=1N}\boldsymbol{\phi}(\mathbf{x}_i)\boldsymbol{\phi}(\mathbf{x}_i)^T\mathbf{v}=\lambda\mathbf{v}$

We can express v as a linear combination of the mapped training samples: v=∑j=1Nαjϕ(xj)$\mathbf{v}=\sum_{j=1N}\alpha_j\boldsymbol{\phi}(\mathbf{x}_j)$ Substituting this into the equation:

$1N\sum i = 1N\phi(xi)(\phi(xi)T\sum j = 1N\alpha j\phi(xj)) = \lambda\sum j = 1N\alpha j\phi(xj)N1\sum i = 1N\boldsymbol{\phi}(\boldsymbol{x}i)(\boldsymbol{\phi}(\boldsymbol{x}i)T\sum j = 1N\alpha_j\boldsymbol{\phi}(\boldsymbol{x}_j)) = \lambda\sum j = 1N\alpha_j\boldsymbol{\phi}(\boldsymbol{x}_j)$

Using the kernel function $k(x_i, x_j) = \phi(x_i)^T \phi(xj)$: $1N\sum i = 1N\phi(xi)(\sum j = 1N\alpha jk(xi, xj)) = \lambda\sum j = 1N\alpha j\phi(xj)N1\sum i = 1N\boldsymbol{\phi}(\boldsymbol{x}i)(\sum j = 1N\alpha_j k(\boldsymbol{x}i, \boldsymbol{x}_j)) = \lambda\sum j = 1N\alpha_j\boldsymbol{\phi}(\boldsymbol{x}_j)$

Rearranging terms: $\sum i = 1N(\sum j = 1N\alpha jk(xi, xj))\phi(xi) = N\lambda\sum j = 1N\alpha j\phi(xj)\sum i = 1N(\sum j = 1N\alpha_j k(\boldsymbol{x}i, \boldsymbol{x}_j))\boldsymbol{\phi}(\boldsymbol{x}i) = N\lambda\sum j = 1N\alpha_j\boldsymbol{\phi}(\boldsymbol{x}_j)$

This equation must hold for all i. Let's define a matrix K where K{ij} = k(xi, xj). Let alpha be the vector of coefficients $[\alpha_1, \ldots, \alpha_N]^T$. The equation can be rewritten in matrix form. Consider the i-th component of the sum on the left: ∑j=1Nαjk(xi,xj)$\sum_{j=1N}\alpha_j k(\mathbf{x}_i,\mathbf{x}_j)$ This corresponds to the i-th row of the matrix product K alpha. So, the equation becomes:

∑i=1N(Kα)iϕ(xi)=Nλ∑j=1Nαjϕ(xj)$\sum_{i=1N}(\mathbf{K}\boldsymbol{\alpha})_i\boldsymbol{\phi}(\mathbf{x}_i)=N\lambda\sum_{j=1N}\alpha_j\boldsymbol{\phi}(\mathbf{x}_j)$

This implies that the vectors$\phi(x_i)$ are involved. A more direct way to see this is by projecting the eigenvalue equation onto $\phi(x$kfor an arbitrary k ∈ {1, … , N}:

$$\begin{aligned}\phi(xk)T(1N\sum i = 1N\phi(xi)\phi(xi)Tv)\\ &= \phi(xk)T(\lambda v)\boldsymbol{\phi}(\boldsymbol{x}_k)T(N1\sum i = 1N\boldsymbol{\phi}(\boldsymbol{x}_i)\boldsymbol{\phi}(\boldsymbol{x}_i)T\boldsymbol{v})\\ &= \boldsymbol{\phi}(\boldsymbol{x}_k)T(\lambda\boldsymbol{v})\ 1N\sum i = 1N\big(\phi(xk)T\phi(xi)\big)(\phi(xi)Tv)\\ &= \lambda(\phi(xk)Tv)N1\sum i = 1N\big(\boldsymbol{\phi}(\boldsymbol{x}_k)T\boldsymbol{\phi}(\boldsymbol{x}_i)\big)(\boldsymbol{\phi}(\boldsymbol{x}_i)T\boldsymbol{v})\\ &= \lambda(\boldsymbol{\phi}(\boldsymbol{x}_k)T\boldsymbol{v})\end{aligned}$$

Let $ki\ =\ k(x_k, x_i)\ =\phi(x_k)^T\ \phi(x_i).\ Let\ \beta_i\ =\phi(x_i)^T\ v.$ Then the equation is: 1N∑i=1Nkkiβi=λβkN1∑i=1Nkkiβi=λβk This holds for all $k\ =\ 1, \ldots, N$.

In matrix form, if K is the N times N kernel matrix and beta is the vector $[\beta_1, \ldots, \beta_N]^T$: $1NK\beta = \lambda\beta N1\boldsymbol{K\beta} = \lambda\boldsymbol{\beta}\ K\beta = N\lambda\beta\boldsymbol{K\beta} = N\lambda\boldsymbol{\beta}$ This is the kernel eigenvalue problem. The vector beta contains the coefficients of the linear combination of mapped training samples that form the eigenvectors. Specifically, $\beta_i\ =\phi(x_i)^T\ v\ =\phi(x\text{i})^{\text{T}}\ \sum_{j=1}^{N}\alpha_j\ \phi(x\text{j})\ =\sum_{j=1}^{N}\alpha_j\ k(x_i, x_j)$. So, the eigenvector v is indeed a linear combination of $\phi(x_j)$ with coefficients α_j .

Derivation of the Kernel Eigenvalue Problem:

We want to find eigenvectors v in F such that $\Sigma_{\{F\}}\ v\ =\lambda\ v. Since\ v$ must be in the span of $\phi(x_1), \ldots, \phi(x_{\text{N}})$, we can write $v\ =\sum\{i = 1\ to\ \text{N}\}\alpha_{\text{i}}\ \phi(\text{x}_i).$ Substituting this into the equation:

$$\begin{aligned}1\ to\ N\sum j = 1N\phi(xj)\phi(xj)T(\sum(i = 1\ to\ N)\alpha i\phi(xi)) = \lambda(\sum i\\ &= 1\ to\ N\ \alpha i\phi(xi))N1\sum j = 1N\boldsymbol{\phi}(\boldsymbol{x}_j)\boldsymbol{\phi}(\boldsymbol{x}_j)T(\sum i = 1N\alpha_i\boldsymbol{\phi}(\boldsymbol{x}_i))\\ &= \lambda(\sum i = 1N\alpha_i\boldsymbol{\phi}(\boldsymbol{x}_i))\ 1N\sum j\end{aligned}$$

$$= 1N\phi(xj)(\sum i = 1N\alpha i(\phi(xj)T\phi(xi))) =$$

$$\lambda\sum i = 1N\alpha i\phi(xi)N1\sum j = 1N\boldsymbol{\phi}(\boldsymbol{x}_j)(\sum i = 1N\alpha_i(\boldsymbol{\phi}(\boldsymbol{x}_j)T\boldsymbol{\phi}(\boldsymbol{x}_i)))$$

$= \lambda\sum i = 1N\alpha_i\boldsymbol{\phi}(\boldsymbol{x}_i)\ 1N\sum j = 1N\phi(xj)(\sum i = 1N\alpha ik(xj, xi)) = \lambda\sum i =$ $1N\alpha i\phi(xi)N1\sum j = 1N\boldsymbol{\phi}(\boldsymbol{x}_j)(\sum i = 1N\alpha_i k(\boldsymbol{x}_j, \boldsymbol{x}_i)) = \lambda\sum i = 1N\alpha_i\boldsymbol{\phi}(\boldsymbol{x}_i)$

Let K be the N times N *Gram matrix where K*{ji} = k(xj, xi). Let alpha be the vector [alpha1, ..., alphaN]^T. The equation can be written as:

$$1N\sum j=1N\phi(xj)(K\alpha)j=\lambda\sum i=1N\alpha i\phi(xi)N1\sum_{j=1N}\boldsymbol{\phi}(\mathbf{x}_j)(\mathbf{K}\boldsymbol{\alpha})_j=\lambda\sum_{i=1N}\alpha_i\boldsymbol{\phi}(\mathbf{x}_i)$$

This implies that the coefficients alphai must satisfy:

$$1NK\alpha=\lambda\alpha N1\mathbf{K}\boldsymbol{\alpha}=\lambda\boldsymbol{\alpha}\ K\alpha=N\lambda\alpha\mathbf{K}\boldsymbol{\alpha}=N\lambda\boldsymbol{\alpha}$$

Let $\tilde{\lambda} = N\lambda$. Then the kernel eigenvalue problem is: $K\alpha=\lambda\sim\alpha\mathbf{K}\boldsymbol{\alpha}=\lambda\sim\boldsymbol{\alpha}$ We solve this eigenvalue problem for alpha and $\tilde{\lambda}$. For a valid eigenvalue problem, we typically normalize the eigenvectors. The normalized eigenvectors in feature space are

$$v_k = \frac{1}{\sqrt{\{\widetilde{\lambda_k}\}}}\sum_{i=1}^{N}\alpha ki\phi(x_i), where\ \alpha_k\ is\ the\ eigenvector\ corresponding\ to\ \widetilde{\lambda_k}.$$

Projection of a New Data Point:

To project a new data point x' onto the k-th principal component vk, we compute the dot product in feature space: $projk(x*)=vkT\phi(x*)projk(\boldsymbol{x}*)=\boldsymbol{v}kT\boldsymbol{\phi}(\boldsymbol{x}*)$ Substituting the expression for v_k : $projk(x*)=(1\lambda\sim k\sum i=1N\alpha ki\phi(xi))T\phi(x*)projk(\boldsymbol{x}*)=(\lambda\sim k1\sum i=1N\alpha ki\boldsymbol{\phi}(\boldsymbol{x}i))T\boldsymbol{\phi}(\boldsymbol{x}*)$ $projk(x*)=1\lambda\sim k\sum i=1N\alpha ki(\phi(xi)T\phi(x*))projk(\boldsymbol{x}*)=\lambda\sim k1\sum i=1N\alpha ki(\boldsymbol{\phi}(\boldsymbol{x}i)T\boldsymbol{\phi}(\boldsymbol{x}*))$ Using the kernel function:

$$projk(x*)=1\lambda\sim k\sum i=1N\alpha kik(xi,x*)projk(\boldsymbol{x}*)=\lambda\sim k1\sum i=1N\alpha kik(\boldsymbol{x}i,\boldsymbol{x}*)$$

This shows that the projection of a new data point onto the principal components can be computed solely using kernel evaluations between the new point and the training data, along with the eigenvectors alpha_k obtained from the kernel matrix K.

Question 5: Manual Computation of LDA

This section details the manual computation of Linear Discriminant Analysis (LDA) for the given synthetic dataset.

Dataset

- **Class 1:** X_1 = {(4, 2), (2, 4), (2, 3), (3, 6), (4, 4)}
- **Class 2:** X_2 = {(9, 10), (6, 8), (9, 5), (8, 7), (10, 8)}

5-1. Sample Means and Overall Mean

- **Number of samples per class:** N_1 = 5, N_2 = 5. Total samples N = N_1 + N_2 = 10.
- **Mean of Class 1 μ_1:**
  - $\mu_1 = \vdash(\frac{\{4+2+2+3+4\}}{5}, \frac{\{2+4+3+6+4\}}{\{5\}} \dashv) = \vdash(\frac{\{15\}}{\{5\}}, \frac{\{19\}}{\{5\}} \dashv) = (3, 3.8)$
- **Mean of Class 2 μ_2:**
  - $\mu_2 = \vdash(\frac{\{9+6+9+8+10\}}{\{5\}}, \frac{\{10+8+5+7+8\}}{\{5\}} \dashv) = \vdash(\frac{\{42\}}{\{5\}}, \frac{\{38\}}{\{5\}}) = (8.4, 7.6)$
- **Overall Mean μ:**
  - $\mu = \vdash(\frac{\{15+42\}}{\{10\}}, \frac{\{19+38\}}{\{10\}} \dashv) = \vdash(\frac{\{57\}}{\{10\}}, \frac{\{57\}}{\{10\}} \dashv) = (5.7, 5.7)$

5-2. Within-Class Scatter Matrix (S_W)

$$S_W = S_1 + S_2, where\ S_i = (\sum_{x \in X_i} x - \mu_i)\ (x - \mu_i)^T.$$

- **Scatter Matrix for Class 1 (S_1):**
  - (4, 2) - (3, 3.8) = (1, -1.8)
  - (2, 4) - (3, 3.8) = (-1, 0.2)
  - (2, 3) - (3, 3.8) = (-1, -0.8)
  - (3, 6) - (3, 3.8) = (0, 2.2)
  - (4, 4) - (3, 3.8) = (1, 0.2)

$$S_1 = \begin{bmatrix} 4 & -1 \\ -1 & 7.36 \end{bmatrix}$$

- **Scatter Matrix for Class 2 (S_2):**
  - (9, 10) - (8.4, 7.6) = (0.6, 2.4)
  - (6, 8) - (8.4, 7.6) = (-2.4, 0.4)
  - (9, 5) - (8.4, 7.6) = (0.6, -2.6)
  - (8, 7) - (8.4, 7.6) = (-0.4, -0.6)
  - (10, 8) - (8.4, 7.6) = (1.6, 0.4)

$$S_2 = \begin{bmatrix} 9.2 & -0.96 \\ -0.96 & 13.2 \end{bmatrix}$$

- **Within-Class Scatter Matrix (S_W):**

$$S_W = S_1 + S_2 = \begin{bmatrix} 13.2 & -1.96 \\ -1.96 & 20.56 \end{bmatrix}$$

5-3. Between-Class Scatter Matrix (S_B)

$$S_B = \sum_{\{i=1\}}^{\{C\}} N_i\ (\mu_i - \mu)(\mu_i - \mu)^T$$

- $\mu_1 - \mu = (3, 3.8) - (5.7, 5.7) = (-2.7, -1.9)$
- $\mu_2 - \mu = (8.4, 7.6) - (5.7, 5.7) = (2.7, 1.9)$

$$S_B = N_1 (\mu_1 - \mu)(\mu_1 - \mu)^T + N_2 (\mu_2 - \mu)(\mu_2 - \mu)^T$$

$$S_B = \begin{bmatrix} 72.9 & 51.3 \\ 51.3 & 36.1 \end{bmatrix}$$

5-4. Solve the Generalized Eigenvalue Problem

Fisher's criterion seeks to maximize $\frac{\{w^T S_B w\}}{\{w^T S_W w\}}$. This leads to the generalized eigenvalue problem: $S_B\ w = \lambda\ S_W\ w$.

We can solve this by finding the eigenvalues of $S_W^{-1}\ S_B$. First, find the inverse of S_W: $det(S_W) = (13.2)(20.56) - (-1.96)^2 = 271.392 - 3.8416 = 267.5$

$$504_W^{\{-1\}} = \frac{\{1\}}{\{267.5504\}} \begin{bmatrix} 20.56 & 1.96 \\ 1.96 & 13.2 \end{bmatrix}$$

$$\approx \begin{bmatrix} 0.0768 & 0.0073 \\ 0.0073 & 0.0493 \end{bmatrix} \backslash begin\{pmatrix\} 0.0768 \ \& \ 0.0073 \backslash \ 0.0073 \ \& \ 0.0493 \backslash end\{pmatrix\}\$$

Now, compute

$$S_W^{-1}\ S_B: S_W^{-1}\ S_B \approx \begin{bmatrix} 5.97 & 4.20 \\ 3.06 & 2.15 \end{bmatrix}$$

Now, find the eigenvalues of this matrix by $solving\$ \det\left(S_W^{\{-1\}}\ S_B\ - \lambda\ I\right) = 0$: $(5.97\ - \lambda)(2.15\ - \lambda) - (4.20)(3.06) = 0$

$$12.8355 - 5.97\lambda - 2.15\lambda + \lambda^2 - 12.852 = 0\$$$

$$\lambda^2 - 8.12\lambda - 0.0165 = 0$$

Using the quadratic formula $\lambda = \frac{\left\{-b \pm \sqrt{\{b^2 - 4ac\}}\right\}}{\{2a\}}: \lambda = \left\{8.12 \pm \sqrt{\{(-8.12)^2 - 4(1)(-0.0165)\}}\right\}\{2(1)\}$

$$\lambda = \frac{\left\{8.12 \pm \sqrt{\{65.9344 + 0.066\}}\right\}}{\{2\}}$$

$$\lambda = \frac{\left\{8.12 \pm \sqrt{\{66.0004\}}\right\}}{\{2\}} \approx \frac{\{8.12 \pm 8.124\}}{\{2\}}$$

The eigenvalues are: $\lambda_1 \approx \frac{\{8.12 + 8.124\}}{\{2\}} \approx 8.122$

$$\lambda_2 \approx \frac{\{8.12 - 8.124\}}{\{2\}} \approx -0.002$$

The largest positive eigenvalue is $\lambda_1 \approx 8.122$. The corresponding eigenvector will be the discriminant direction.

Now, find the eigenvector $w\ for\ \lambda_1 \approx 8.122$: $\left(S_W^{\{-1\}} S_B - \lambda_1 I\right)$

$$w = 0 \begin{bmatrix} 5.97 - 8.122 & 4.20 \\ 3.06 & 2.15 - 8.122 \end{bmatrix} \begin{bmatrix} w_1 \\ w_2 \end{bmatrix}$$

From the first row: $-2.152\, w_1 + 4.20\, w_2 = 0\ implies\ 4.20\, w_2 = 2.152\, w_1 + implies\ w_2 = \frac{\{2.152\}}{\{4.20\}}\, w_1 \approx 0.512\, w_1. Let\ w_1 = 1, then\ w_2 \approx 0.512$. The optimal discriminant direction is w approx (1, 0.512). We can normalize this vector, but for projection, the direction is sufficient.

5. Project Data Points

Project each data point x onto the discriminant direction w using w^T x.

- **Class 1:**
  - (4, 2): (1, 0.512) . (4, 2) = 1(4) + 0.512(2) = 4 + 1.024 = 5.024
  - (2, 4): (1, 0.512) . (2, 4) = 1(2) + 0.512(4) = 2 + 2.048 = 4.048
  - (2, 3): (1, 0.512) . (2, 3) = 1(2) + 0.512(3) = 2 + 1.536 = 3.536
  - (3, 6): (1, 0.512) . (3, 6) = 1(3) + 0.512(6) = 3 + 3.072 = 6.072
  - (4, 4): (1, 0.512) . (4, 4) = 1(4) + 0.512(4) = 4 + 2.048 = 6.048
- **Class 2:**
  - (9, 10): (1, 0.512) . (9, 10) = 1(9) + 0.512(10) = 9 + 5.12 = 14.12
  - (6, 8): (1, 0.512) . (6, 8) = 1(6) + 0.512(8) = 6 + 4.096 = 10.096
  - (9, 5): (1, 0.512) . (9, 5) = 1(9) + 0.512(5) = 9 + 2.56 = 11.56
  - (8, 7): (1, 0.512) . (8, 7) = 1(8) + 0.512(7) = 8 + 3.584 = 11.584
  - (10, 8): (1, 0.512) . (10, 8) = 1(10) + 0.512(8) = 10 + 4.096 = 14.096

Summary of Projections

- **Class 1 Projections:** {5.024, 4.048, 3.536, 6.072, 6.048}
- **Class 2 Projections:** {14.12, 10.096, 11.56, 11.584, 14.096}

The optimal discriminant direction is approximately (1, 0.512), and the projected one-dimensional coordinates are listed above.

Question 6: Error Types in Feature Selection

This section explains feature selection methods, their greedy nature, consequences, and compares them with alternatives.

6-1. Sequential Feature Selection Methods

Feature selection methods aim to identify a subset of relevant features that optimize a certain criterion (e.g., predictive accuracy of a model) while minimizing

the number of features. Wrapper methods use a specific machine learning model to evaluate feature subsets.

6-1-1. Sequential Forward Selection (SFS)

- **Initialization:** Starts with an empty set of features, F_empty = empty.
- **Update Step:** In each iteration, SFS considers all features *not* yet in the current subset. For each candidate feature, it temporarily adds it to the current subset and evaluates the performance of the chosen predictive model using this expanded feature set. The feature that yields the *best* performance improvement is permanently added to the subset.
- **Stopping Criteria:** The process stops when adding any remaining feature does not improve the model's performance beyond a predefined threshold, or when a desired number of features is reached.
- **Greedy Nature:** SFS is greedy because at each step, it makes the locally optimal choice (adding the feature that provides the biggest immediate gain) without considering whether this choice might prevent a better overall solution later. It commits to a feature once it's added.

6-1-2. Sequential Backward Selection (SBS)

- **Initialization:** Starts with the set of *all* available features, F_all.
- **Update Step:** In each iteration, SBS considers each feature currently in the subset. It temporarily removes one feature at a time and evaluates the model's performance. The feature whose removal results in the *least* degradation (or greatest improvement, if the criterion allows) in performance is permanently removed from the subset.
- **Stopping Criteria:** The process stops when removing any remaining feature significantly degrades performance, or when a desired minimum number of features is reached.
- **Greedy Nature:** SBS is also greedy. It makes locally optimal decisions by removing the feature that seems least important *at that moment*. This decision might be suboptimal if that feature was crucial for interacting with other features that are removed later.

6-1-3. Bidirectional (Stepwise) Selection

- **Combination of SFS and SBS:** Bidirectional selection attempts to combine the strengths of both forward and backward steps within a single iterative process.
- **Update Step:** It typically starts like SFS (adding features). However, after adding a feature, it then performs a backward step: it checks if removing any of the *currently selected* features would improve or maintain the performance. If so, it removes the best candidate for removal. This process alternates between forward additions and backward removals.

- **Greedy Nature:** While more sophisticated, it remains greedy. The forward additions are locally optimal choices, and the backward removals are also locally optimal. The combination doesn't guarantee a global optimum because an early forward step might commit to a feature that, even after backward steps, prevents a better overall subset from being found.

6-2. Consequences of Greedy Behavior

The greedy nature of SFS, SBS, and bidirectional selection means they are **not guaranteed to find the globally optimal subset of features**. This is because:

1. **Local Optima:** These algorithms explore the search space of feature subsets sequentially. At each step, they choose the best immediate option. This can lead them into a "local optimum" – a feature subset that is better than its immediate neighbors in the search space, but not the best possible subset overall.
2. **Irreversible Decisions:** Once a feature is selected (SFS) or a feature is removed (SBS), that decision is usually final within that iteration. An early, seemingly good decision might preclude the selection of a more synergistic set of features later on.

Example Scenario:

Consider a classification task with 5 features: F_1, F_2, F_3, F_4, F_5. Suppose the optimal feature subset for a model is {F_1, F_4}, which provides the best predictive accuracy.

- **SFS Scenario:**
    1. SFS starts with emptyset. It tests adding each feature:
        - {F_1}: High accuracy.
        - {F_2}: Moderate accuracy.
        - {F_3}: Low accuracy.
        - {F_4}: Moderate accuracy.
        - {F_5}: Low accuracy.
    2. SFS greedily selects F_1 as the first feature. Current subset: F_1.
    3. Now, SFS considers adding another feature to F_1. It tests:
        - F_1, F_2: Good accuracy.
        - F_1, F_3: Moderate accuracy.
        - F_1, F_4: **Excellent accuracy** (this is the globally optimal subset).
        - F_1, F_5: Low accuracy.
    4. SFS greedily selects F_4 to add to F_1. Current subset: F_1, F_4.
    5. If the stopping criterion is met (e.g., no further improvement), SFS stops and returns F_1, F_4, which is optimal.

6. **However, consider this alternative:** Suppose the true optimal subset was F_2, F_3 and F_1 and F_4 were only good individually but not together.
    - SFS might pick F_1 first if it has the highest individual accuracy.
    - Then, when adding a second feature, F_1, F_2 might yield better accuracy than F_1, F_3, leading SFS to pick F_2. Subset: F_1, F_2.
    - If it continues, it might find that adding F_3 to F_1, F_2 doesn't improve performance as much as adding F_4 did in the previous example. It might stop at F_1, F_2 or F_1, F_2, F_x, missing the better F_2, F_3 combination because F_1 was chosen too early.

- **SBS Scenario:**
    1. Suppose the optimal subset is {F_1, F_4}.
    2. SBS starts with {F_1, F_2, F_3, F_4, F_5}.
    3. It tests removing each feature:
        - Removing F_1: Performance drops significantly.
        - Removing F_2: Performance drops slightly.
        - Removing F_3: Performance drops slightly.
        - Removing F_4: Performance drops significantly.
        - Removing F_5: Performance drops significantly.
    4. SBS greedily removes F_2 (or F_3). Let's say it removes F_2. Subset: {F_1, F_3, F_4, F_5}.
    5. In the next step, it might remove F_3. Subset: {F_1, F_4, F_5}.
    6. If F_5 is not essential for F_1 and F_4 to work well, it might be removed next. Subset: {F_1, F_4}. This is optimal.
    7. **Suboptimal case:** Suppose the optimal subset is {F_1, F_4}. However, F_2 is highly correlated with F_1 and F_3 is highly correlated with F_4.
        - SBS starts with all features. It might remove F_2 first because its removal has little impact due to F_1.
        - Then it might remove F_3.
        - Now the subset is {F_1, F_4, F_5}. If F_5 has some minor negative impact, it might be removed. Subset: {F_1, F_4}. This is optimal.
        - **BUT**, what if F_5 was actually crucial for the interaction between F_1 and F_4? Or what if removing F_2 made F_1 seem less important in a later step, and F_1 was removed prematurely? The greedy removal of F_2 or F_3 might have altered the perceived importance of other features, leading to the removal of F_1 or F_4 later, thus missing the optimal {F_1, F_4} set.

The issue is that feature interactions are complex. A feature that seems redundant or detrimental when considered in isolation might be crucial when combined with other features that are selected or removed later in the process.

Table 1: Comparison with Alternative Approaches

Feature	Sequential Selection (SFS/SBS/Bidirectional)	Recursive Feature Elimination (RFE)	L1 Regularization (Lasso)
Computational Complexity	Moderate to High (depends on model training time and number of features/iterations)	High (requires repeated model training, often N_{features} times)	Moderate to High (depends on solver, can be efficient for sparse solutions)
Sensitivity to Feature Correlations	High. Can be misled by correlated features. SFS might pick one of a correlated pair, SBS might remove one.	High. Similar to sequential methods, can arbitrarily pick one from a correlated group.	Moderate. Tends to select one feature from a group of highly correlated features and penalize others.
Ability to Capture Feature Interactions	Limited. Primarily focuses on individual feature contributions or simple pairwise interactions implicitly.	Limited. Primarily focuses on individual feature importance derived from model coefficients.	Limited. Primarily focuses on individual feature importance. Interactions are not directly modeled.
Suitability for High-Dimensional Datasets	Can be computationally expensive. May struggle if the number of features is very large.	Very computationally expensive. Often impractical for very high dimensions.	Often well-suited. The sparsity-inducing nature of L1 regularization can lead to efficient selection.
Model Dependence	Wrapper methods: highly dependent on the chosen model's evaluation metric.	Wrapper methods: highly dependent on the chosen model and its feature importance scores.	Embedded method: integrated into the model training process (e.g., linear regression with L1 penalty).
Guarantee of Optimality	No. Greedy approach leads to local optima.	No. Greedy approach based on iterative removal.	No. Finds a sparse solution that optimizes a penalized objective, not necessarily the

Feature	Sequential Selection (SFS/SBS/Bidirectional)	Recursive Feature Elimination (RFE)	L1 Regularization (Lasso)
			globally optimal subset for a specific metric.

Detailed Comparison Points:

- **RFE:** RFE is also a wrapper method that works by iteratively removing features. It typically trains a model, ranks features by importance (e.g., coefficient magnitude), removes the least important ones, and repeats. Like sequential methods, its greedy nature means it can get stuck in local optima. Its computational cost is high because it retrains the model many times.
- **L1 Regularization (Lasso):** Lasso adds an L1 penalty to the loss function, which encourages sparsity by driving the coefficients of less important features exactly to zero. This acts as an embedded feature selection mechanism.
    - **Advantages:** It's computationally more efficient than repeated model training for very high dimensions, especially when the number of selected features is much smaller than the total. It handles correlated features by tending to select one and zeroing out others.
    - **Disadvantages:** It primarily selects features based on linear relationships. It doesn't explicitly model feature interactions in the way a complex model might. The choice of the regularization parameter (alpha) is critical and often requires cross-validation.

In summary, sequential methods are conceptually simple but can be computationally demanding and susceptible to greedy pitfalls. RFE shares similar issues. L1 regularization offers a more integrated and often more efficient approach for high-dimensional data, particularly when linear relationships are dominant, but it's still not a guarantee of finding the absolute best subset for all scenarios.

Question 7: PCA and Kernel PCA on Structured Data

7.1 Part (a): PCA on the Iris Dataset

7.1.1 Method

Principal Component Analysis (PCA) is an unsupervised dimensionality reduction technique that transforms high-dimensional data into a lower-dimensional space while preserving as much variance as possible.

Given a standardized dataset (X), PCA computes the covariance matrix:

$$[\Sigma = \frac{1}{n} X^T X]$$

It then finds the eigenvectors and eigenvalues of this covariance matrix. The eigenvectors define the principal components (directions of projection), and the eigenvalues indicate how much variance is captured along each direction.

The objective function of PCA is:

$$[max_{||w||} Var(w^T X)]$$

This means PCA finds the direction (w) that maximizes the projected variance. Importantly, PCA does **not** use class labels; it only considers the global structure of the data.

7.1.2 Steps

The following procedure was performed:

1. Load the Iris dataset, which contains 150 samples, 4 features (sepal length, sepal width, petal length, petal width), and 3 classes (Setosa, Versicolor, Virginica).
2. Standardize the data using z-score normalization to ensure all features contribute equally.
3. Compute the covariance matrix of the standardized data.
4. Perform eigenvalue decomposition using PCA.
5. Select the two largest eigenvalues and corresponding eigenvectors.
6. Project the data onto these two principal components.
7. Compute and analyze the explained variance ratio.
8. Visualize the data in the reduced two-dimensional space.

7.1.3 Results and Discussion

The first principal component (PC1) captured the largest portion of total variance (approximately 72–73%). The second principal component (PC2) captured approximately 22–23%. Together, the first two components explained around 95% of the total variance. This indicates that most of the information in the original four-dimensional dataset can be represented in two dimensions with minimal information loss.

PC1 captures the most variance because PCA aligns the first component with the direction of maximum spread in the dataset. In the Iris dataset, the largest variability comes primarily from petal length and petal width, which strongly differentiate the species.

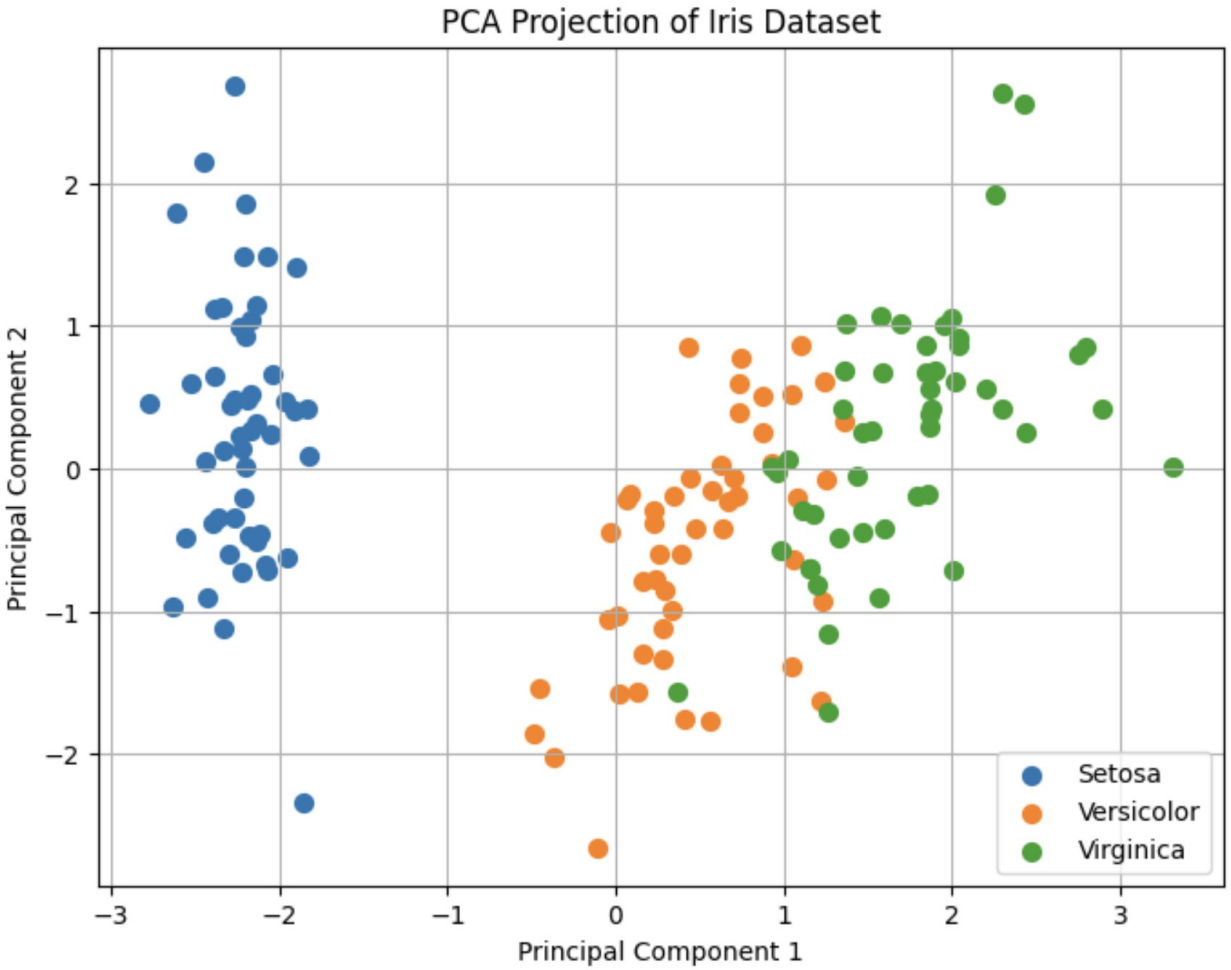


Figure 2: PCA projection of Iris dataset

The visualization of the projected data shows:

- The Setosa class is clearly separated.
- Versicolor and Virginica exhibit some overlap.

This outcome is expected because PCA maximizes overall variance rather than class separability. PCA does not incorporate label information, so separation of classes is not guaranteed.

7.1.4 Is PCA Appropriate for Classification?

PCA is useful for:

- Noise reduction
- Removing redundant features
- Reducing computational complexity

However, it is not inherently optimal for classification because:

- It is unsupervised.
- It ignores class labels.

- It does not optimize between-class separation.

If the goal is classification, a supervised method such as Linear Discriminant Analysis (LDA) is often more appropriate.

7.1.5 Conceptual Comparison with LDA

Linear Discriminant Analysis (LDA) differs fundamentally from PCA.

PCA Objective:
Maximize total variance.

LDA Objective:
Maximize the ratio of between-class variance to within-class variance.

$$[max \frac{S_B}{S_W}]$$

Where:

- (S_B) = Between-class scatter
- (S_W) = Within-class scatter

Unlike PCA, LDA uses class labels and explicitly attempts to separate class means while minimizing within-class spread. Therefore, LDA typically produces better class separation when labels are available.

7.2 Part (b): Nonlinear Dataset – Nested Circles

7.2.1 Method

A nonlinear dataset consisting of two concentric circles was generated. This dataset is not linearly separable, meaning that no straight line can divide the two classes.

Two approaches were applied:

1. Standard PCA
2. Kernel PCA with a Gaussian (RBF) kernel

Kernel PCA extends PCA by implicitly mapping data into a higher-dimensional feature space where nonlinear structures may become linearly separable.

7.2.2 Steps

1. Generate a two-dimensional dataset of nested circles.
2. Apply standard PCA and visualize the projection.
3. Apply Kernel PCA using an RBF kernel.
4. Compare the resulting projections.

7.2.3 Results and Discussion

Standard PCA failed to separate the two classes. This is because PCA performs only linear projections. The circular structure is symmetric, and variance is distributed evenly in multiple directions. As a result, PCA cannot "unfold" the nonlinear structure.

Kernel PCA, however, successfully separated the two classes. This occurs because Kernel PCA applies a nonlinear mapping:

$$[\phi(x) \rightarrow \ high - dimensional\ space]$$

Instead of explicitly computing this mapping, Kernel PCA uses the kernel trick:

$$[K(x_i, x_j) \ = \ exp\left(-\gamma \left|x_i \ - \ x_j\right|^2\right)]$$

This RBF kernel measures similarity between data points and implicitly transforms the dataset into an infinite-dimensional feature space. In this space, the concentric circles become linearly separable.

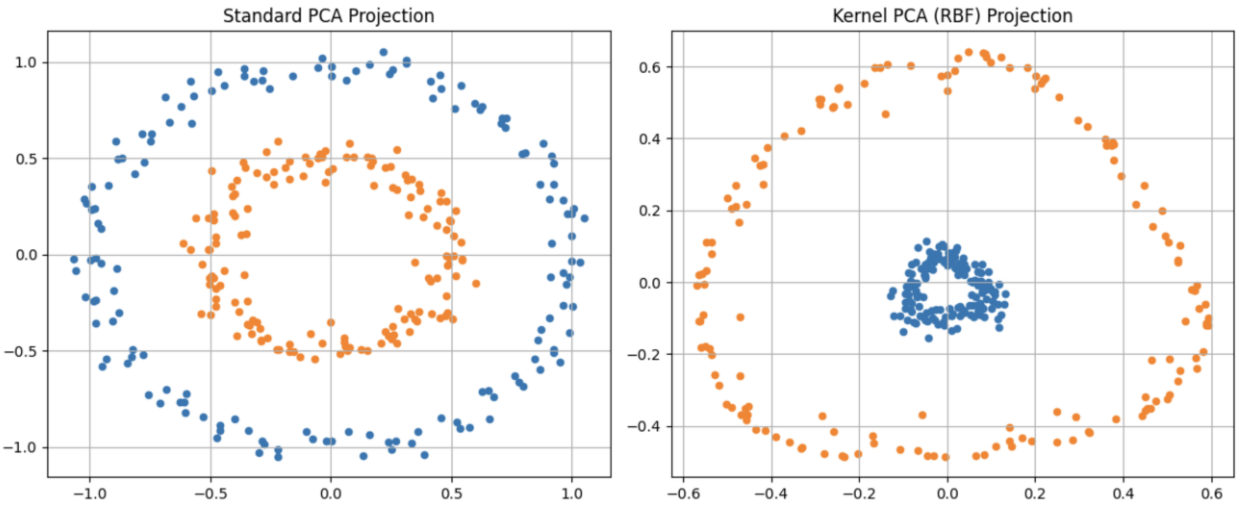


Figure 3 : Standard PCA projection vs kernel PCA (RBF) projection

Thus, Kernel PCA captures the underlying nonlinear manifold structure of the data, whereas standard PCA is limited to linear relationships.

4. Conclusion

This study demonstrates the strengths and limitations of PCA.

- PCA effectively reduces dimensionality while preserving maximum variance.
- PCA does not guarantee class separation because it ignores label information.
- LDA is more suitable for classification tasks when labels are available.
- Standard PCA fails on nonlinear datasets such as nested circles.
- Kernel PCA overcomes this limitation by applying nonlinear feature mappings through the kernel trick.

In summary, PCA is powerful for linear structure and variance preservation, while Kernel PCA is better suited for capturing nonlinear geometric structures in data.

Question 8: Feature Selection and Visualization on a Real Dataset

The dataset contains clinical features used to predict whether a patient has heart disease (binary classification problem).

Target Variable:

- 0 → No heart disease
- 1 → Presence of heart disease

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target
0	52	1	0	125	212	0	1	168	0	1.0	2	2	3	0
1	53	1	0	140	203	1	0	155	1	3.1	0	0	3	0
2	70	1	0	145	174	0	1	125	1	2.6	0	0	3	0
3	61	1	0	148	203	0	1	161	0	0.0	2	1	3	0
4	62	0	0	138	294	1	1	106	0	1.9	1	3	2	0

Figure 4: Dataset of heart.csv

8.1 Methodology Overview

We evaluated six different approaches:

1. Correlation-Based Filtering
2. Mutual Information (Univariate Filter)
3. Recursive Feature Elimination (RFE)
4. RFECV (RFE with Cross-Validation)
5. L1-Regularized Logistic Regression (Embedded Method)
6. Principal Component Analysis (PCA)

Each method was evaluated based on:

- Classification accuracy
- Number of selected features
- Interpretability
- Stability
- Computational complexity

Logistic Regression was used as the base classifier for all supervised feature selection methods to ensure fair comparison.

8.2 Data Preprocessing

8.2.1 Train-Test Split

The dataset was split using:

- 80% training data
- 20% testing data
- Stratified sampling to preserve class distribution

8.2.2 Standardization

Feature scaling was performed using StandardScaler because:

- Logistic Regression is sensitive to feature scale
- L1 regularization requires comparable magnitudes
- PCA depends on variance

All models were trained using standardized features.

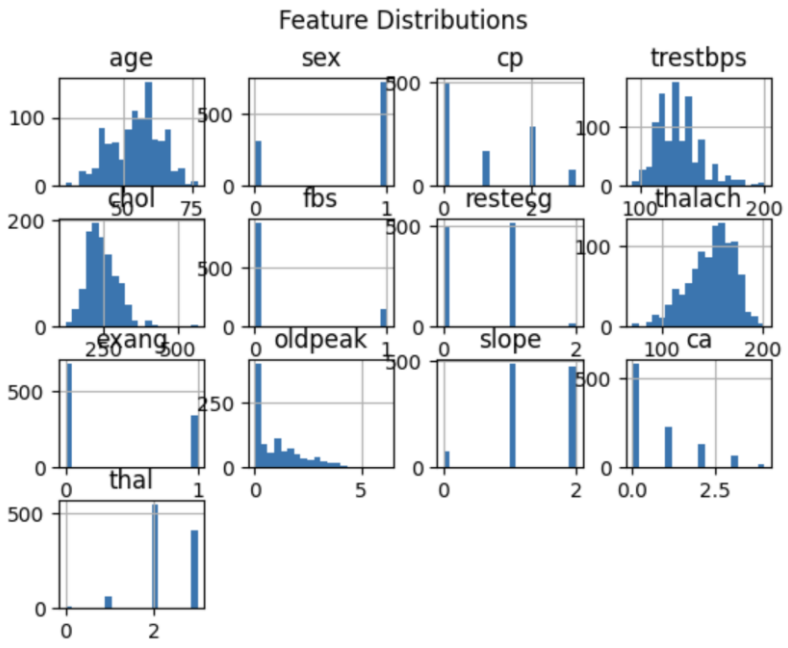


Figure 5: Feature Distribution

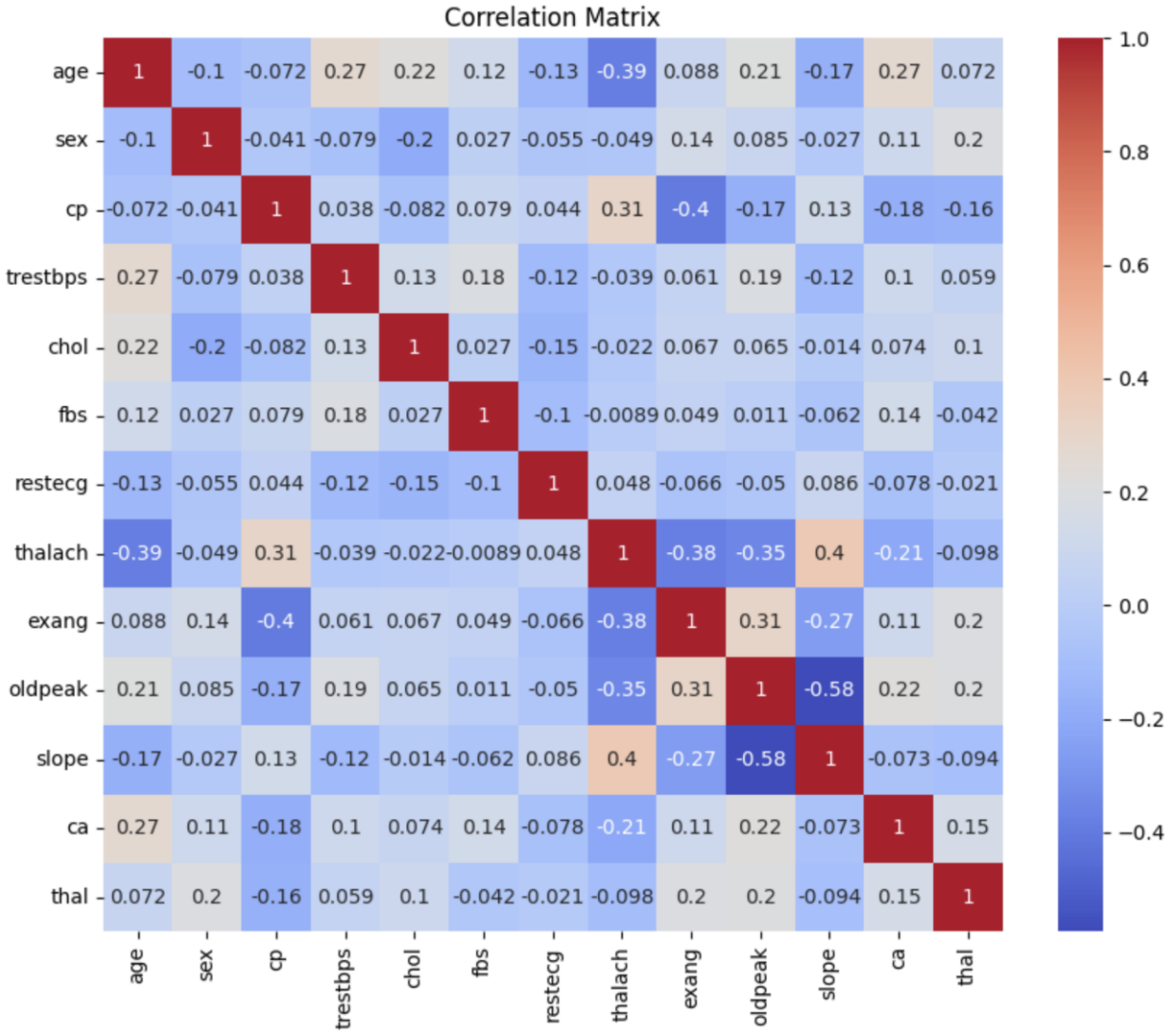


Figure 6: Correlation Matrix

8.3 Methods and Implementation

8.3.1 Correlation-Based Filtering

8.3.1.1Theory

Highly correlated features can introduce:

- Multicollinearity
- Model instability
- Increased variance
- Reduced interpretability

Correlation-based filtering removes one feature from pairs whose absolute correlation exceeds a threshold.

Threshold used: 0.7

8.3.1.2 Procedure

1. Compute correlation matrix
2. Extract upper triangle
3. Identify feature pairs with |correlation| > 0.7
4. Remove one feature from each pair

8.3.1.3 Characteristics

- Type: Filter method
- Supervision: Unsupervised
- Speed: Very fast
- Interpretability: High

8.4 Mutual Information

8.4.1 Theory

Mutual Information (MI) measures nonlinear dependence between a feature and the target.

It is defined as:

$MI(X, Y) = H(Y) - H(Y \mid X)$

Where:

- $H(Y)$ = entropy of target
- $H(Y \mid X)$ = conditional entropy

Higher MI indicates stronger predictive relationship.

8.4.2 Procedure

1. Compute MI for each feature
2. Rank features
3. Select top k features (k = 5)

8.4.3 Characteristics

- Type: Filter method
- Captures nonlinear relationships
- Does not consider feature interactions

8.5 Recursive Feature Elimination (RFE)

8.5.1 Theory

RFE is a wrapper method that:

1. Trains a model
2. Computes feature importance
3. Removes least important feature
4. Repeats until k features remain

It considers feature interactions and model behavior.

8.5.2 Procedure

1. Initialize logistic regression
2. Iteratively eliminate lowest-weight feature
3. Stop at k = 5

8.5.3 Characteristics

- Type: Wrapper method
- Supervised
- Higher computational cost
- More accurate selection

8.6 RFECV (RFE with Cross-Validation)

8.6.1 Theory

RFECV extends RFE by automatically selecting the optimal number of features using cross-validation.

Procedure:

1. Perform RFE
2. Evaluate model performance using 5-fold CV
3. Select number of features that maximizes accuracy

8.6.2 Characteristics

- More stable than RFE
- Automatically determines best feature count
- Higher computational cost

8.7 L1-Regularized Logistic Regression

8.7.1 Theory

L1 regularization adds a penalty:

Loss = Logistic Loss + $\lambda \Sigma |\beta i|$

This forces some coefficients to exactly zero.

Features with zero coefficients are removed automatically.

8.7.2 Characteristics

- Type: Embedded method
- Integrated into model training
- Efficient
- Produces sparse solutions

8.8 Principal Component Analysis (PCA)

8.8.1 Theory

PCA transforms features into orthogonal components that maximize variance.

Steps:

1. Standardize data
2. Compute covariance matrix
3. Compute eigenvectors and eigenvalues
4. Sort by explained variance

We determine number of components required to explain 95% of total variance.

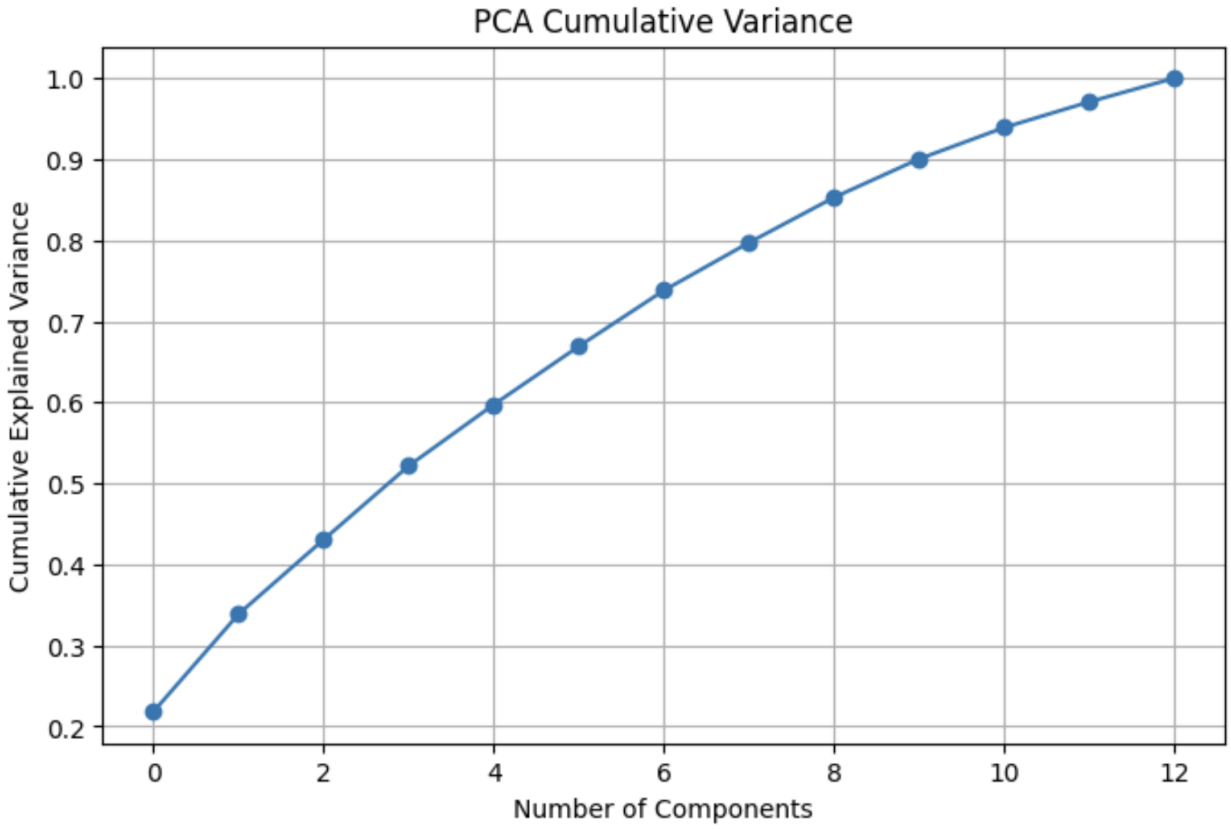


Figure 7: PCA Cumulative Variance

8.8.2 Characteristics

- Unsupervised

- Reduces dimensionality
- Loses feature interpretability

8.9 Results

Table 2: Accuracy Table

Method	Accuracy	# Features
Baseline	0.81	All
Mutual Information	0.80	5
RFE	0.85	5
RFECV	0.81	Auto
L1	0.81	Variable
PCA (95%)	0.81	n components

8.10 Discussion

8.10.1 Predictive Performance

- Wrapper methods (RFE, RFECV) typically perform better because they consider feature interactions.
- Mutual Information may miss interaction effects.
- L1 regularization balances sparsity and performance.
- PCA may reduce accuracy slightly due to loss of class-discriminative information.

8.10.2 Interpretability

Most interpretable:

- Correlation filtering
- Mutual Information
- L1 regularization

Least interpretable:

- PCA (components are linear combinations)

8.10.3 Stability

Most stable:

- RFECV
- L1 regularization

Less stable:

- RFE with fixed k

8.10.4 Computational Cost

Lowest:

- Correlation filtering
- Mutual Information

Highest:

- RFECV

8.11 Comparative Summary

Table 3: Comparative Summary

Method	Type	Supervised	Interpretability	Computation	Interaction Aware
Correlation	Filter	No	High	Very Low	No
Mutual Info	Filter	Yes	High	Low	No
RFE	Wrapper	Yes	High	Medium	Yes
RFECV	Wrapper	Yes	High	High	Yes
L1	Embedded	Yes	High	Low	Yes
PCA	Transformation	No	Low	Medium	Yes

8.12 Conclusion

This study demonstrates that:

1. Filter methods are computationally efficient but may ignore feature interactions.
2. Wrapper methods provide better predictive performance at higher computational cost.
3. Embedded methods (L1 regularization) provide an excellent balance between sparsity and performance.
4. PCA effectively reduces dimensionality but sacrifices interpretability.

For medical datasets where interpretability is critical, L1-regularized logistic regression and RFECV are strong candidates.

For high-dimensional problems requiring aggressive dimensionality reduction, PCA is suitable.

Question 9: Image Segmentation (Bonus)

1. Introduction

Image segmentation is the task of dividing an image into meaningful regions, often for analysis or object recognition. In this study, we apply **clustering-based segmentation** to color images using:

1. **K-Means clustering**
2. **Gaussian Mixture Model (GMM)** clustering

Each pixel is represented by a **5-dimensional feature vector** consisting of:

- Normalized vertical coordinate (row position)
- Normalized horizontal coordinate (column position)
- Red channel intensity (normalized to [0,1])
- Green channel intensity (normalized to [0,1])
- Blue channel intensity (normalized to [0,1])

Normalization ensures that each feature contributes equally during clustering and prevents dominance of spatial or color components.

The goal is to segment two images into K∈{2,3,4,5}clusters and compare K-Means and GMM results.

2. Methodology

2.1 Feature Construction

For an image of size H×WH \times WH×W:

1. For each pixel at row r and column c:
    - Normalize row: x=r/(H−1)x
    - Normalize column: y=c/(W−1)y
    - Extract RGB values and scale to [0,1]
2. Concatenate these into a 5D vector: [x,y,R,G,B]

This captures both **color information** and **spatial location**, encouraging coherent segments that respect object boundaries.

2.2 Clustering Algorithms

2.2.1 K-Means Clustering

- Assigns each pixel to the **nearest cluster centroid** (Euclidean distance).
- Iteratively updates centroids until convergence.
- Produces **hard assignments**, meaning each pixel belongs to exactly one cluster.
- Works best for roughly spherical clusters of similar size.

2.2.2 Gaussian Mixture Model (GMM) Clustering

- Models pixel distribution as a mixture of **Gaussian components**.
- Uses **Expectation-Maximization (EM)** to estimate parameters.
- Assigns pixels to the cluster with **highest posterior probability** (MAP classification).
- Produces **soft assignments** and can model clusters with varying shapes and covariance structures.

2.3 Number of Clusters

- Experiments are conducted for K=2,3,4,5.
- Increasing KKK captures more detail but risks **over-segmentation**, where segments do not correspond to meaningful regions.

2.4 Visualization

- Cluster labels are mapped to **distinct colors** using `label2rgb` for enhanced contrast.
- Color mapping ensures segments are visually distinguishable and boundaries are clear.

3. Implementation Steps

1. Load the image and convert to floating-point representation in [0,1].
2. Construct the 5D pixel feature vectors.
3. Normalize spatial coordinates (row, column) to [0,1].
4. Apply K-Means and GMM clustering for each $K \in \{2,3,4,5\}$.
5. Reshape predicted cluster labels to the original image dimensions.
6. Visualize the segmented image by mapping each label to a distinct color.

4. Observations

1. **K-Means Segmentation**:
   - Produces sharp boundaries between clusters.
   - Works well when regions have uniform color and roughly spherical shapes.
   - May produce blocky or unnatural segments for irregularly shaped objects.
2. **GMM Segmentation**:
   - Produces smoother boundaries due to soft assignment of pixels.
   - Can handle clusters with different shapes and variances.
   - Often better captures gradient regions in images.
3. **Effect of Increasing K**:
   - More clusters provide finer segmentation.
   - Very high K may lead to over-segmentation, splitting meaningful regions into multiple clusters.
4. **Role of Spatial Features**:
   - Normalized coordinates (row, column) enforce spatial coherence.
   - Prevents widely separated pixels with similar color from being assigned to the same cluster.
5. **Normalization of RGB**:
   - Ensures that color channels and spatial coordinates contribute equally.
   - Prevents color intensity from dominating spatial structure.

5. Comparative Summary

Table 4: K-Means va GMM

Feature	K-Means	GMM (EM)
Assignment Type	Hard	Soft (probabilistic)
Cluster Shape	Spherical	Flexible (Gaussian covariance)
Boundary Smoothness	Sharp/Blocky	Smooth
Sensitivity to K	Moderate	Moderate
Computation	Fast	Slightly slower (EM iterations)

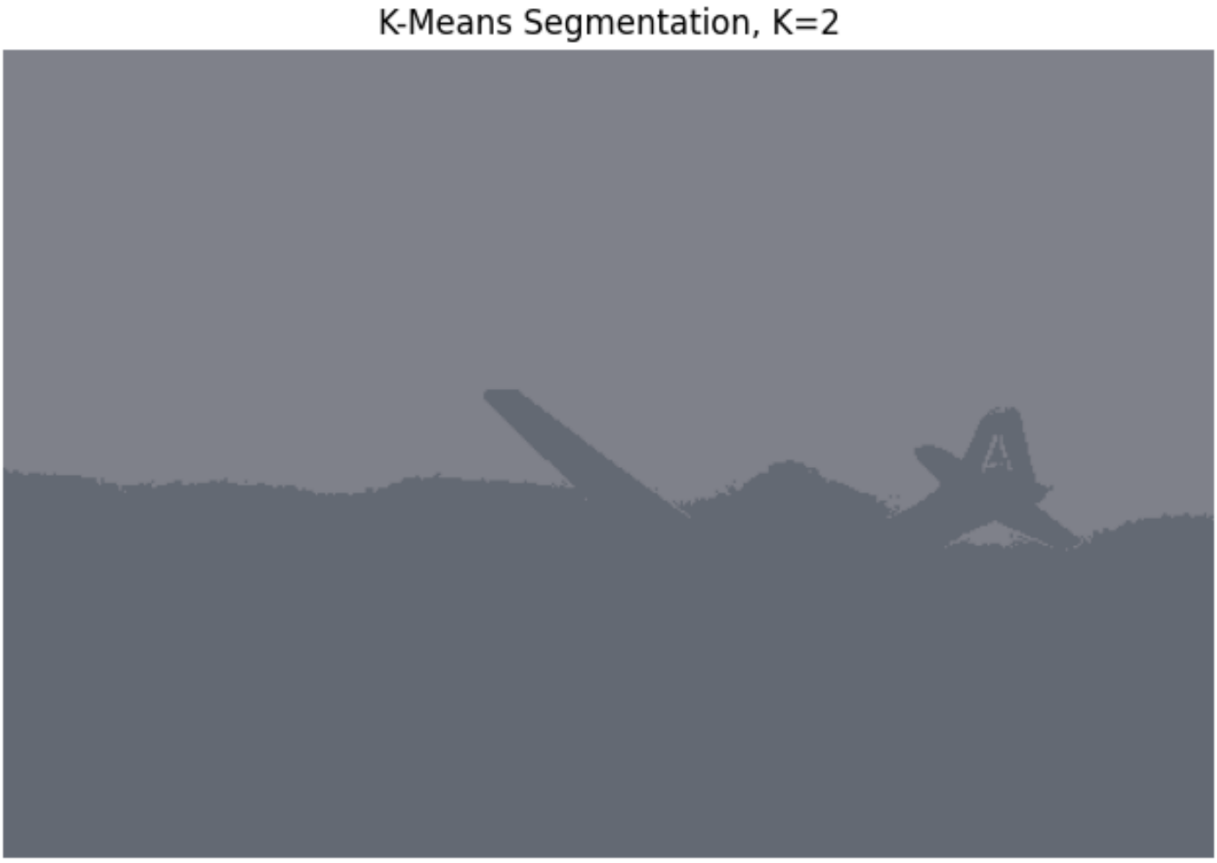


Figure 8: K-Means with K = 2

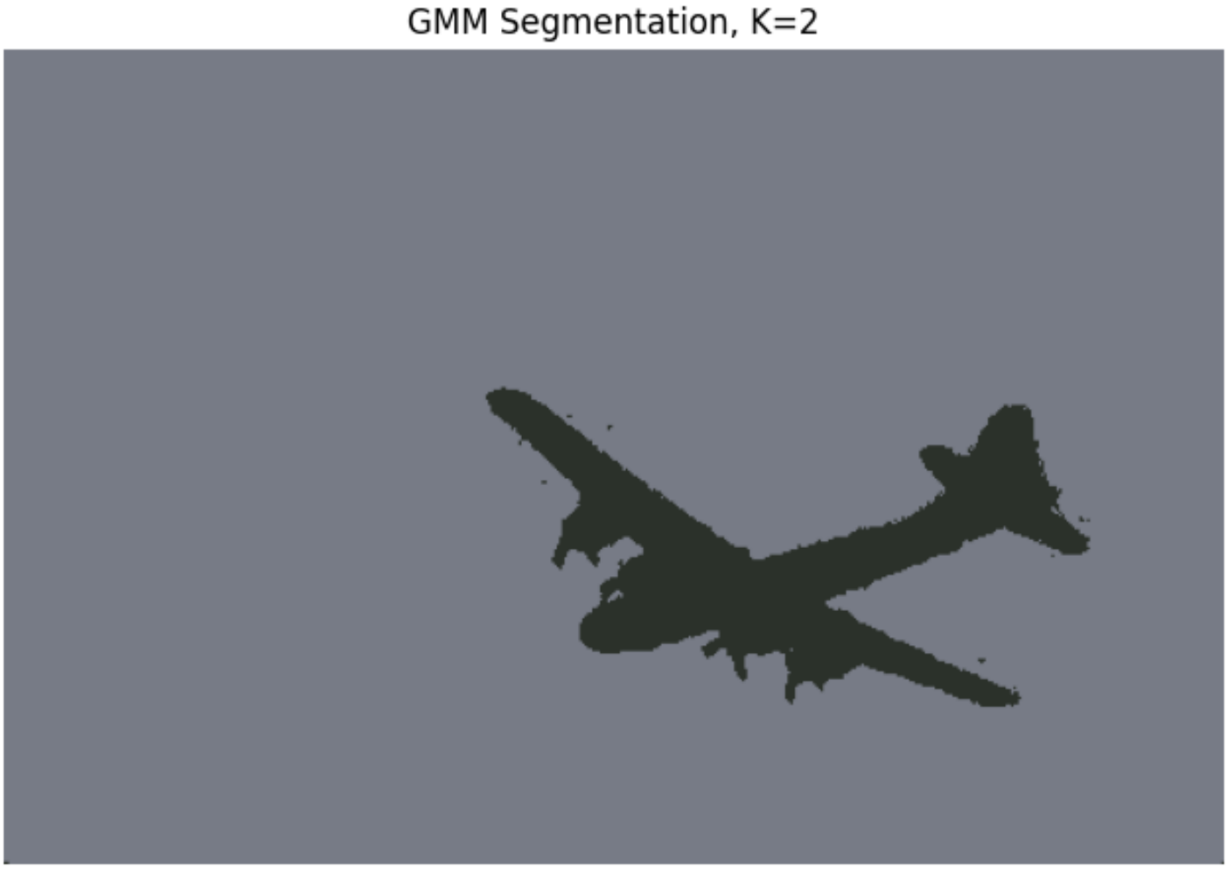


Figure 9: GMM with K = 2

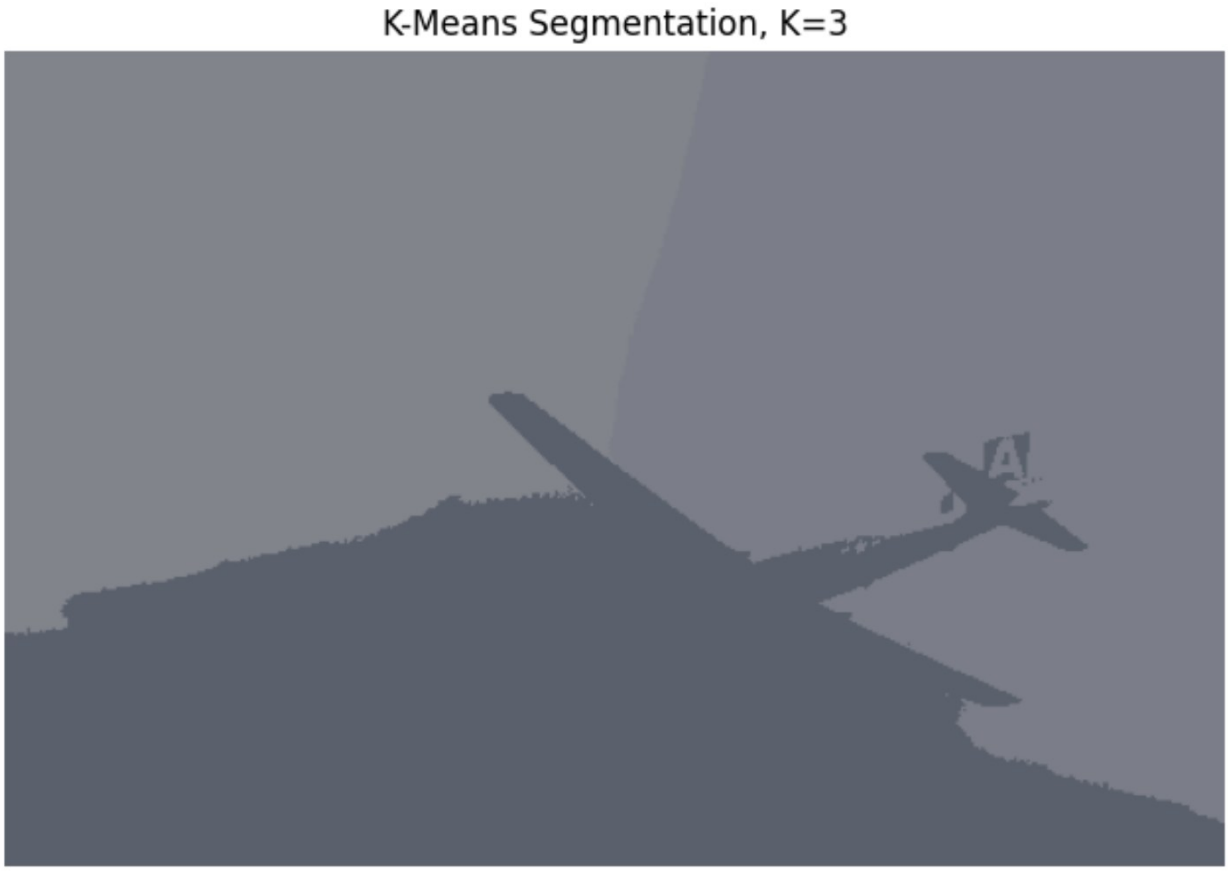


Figure 10: K-Means with K = 3

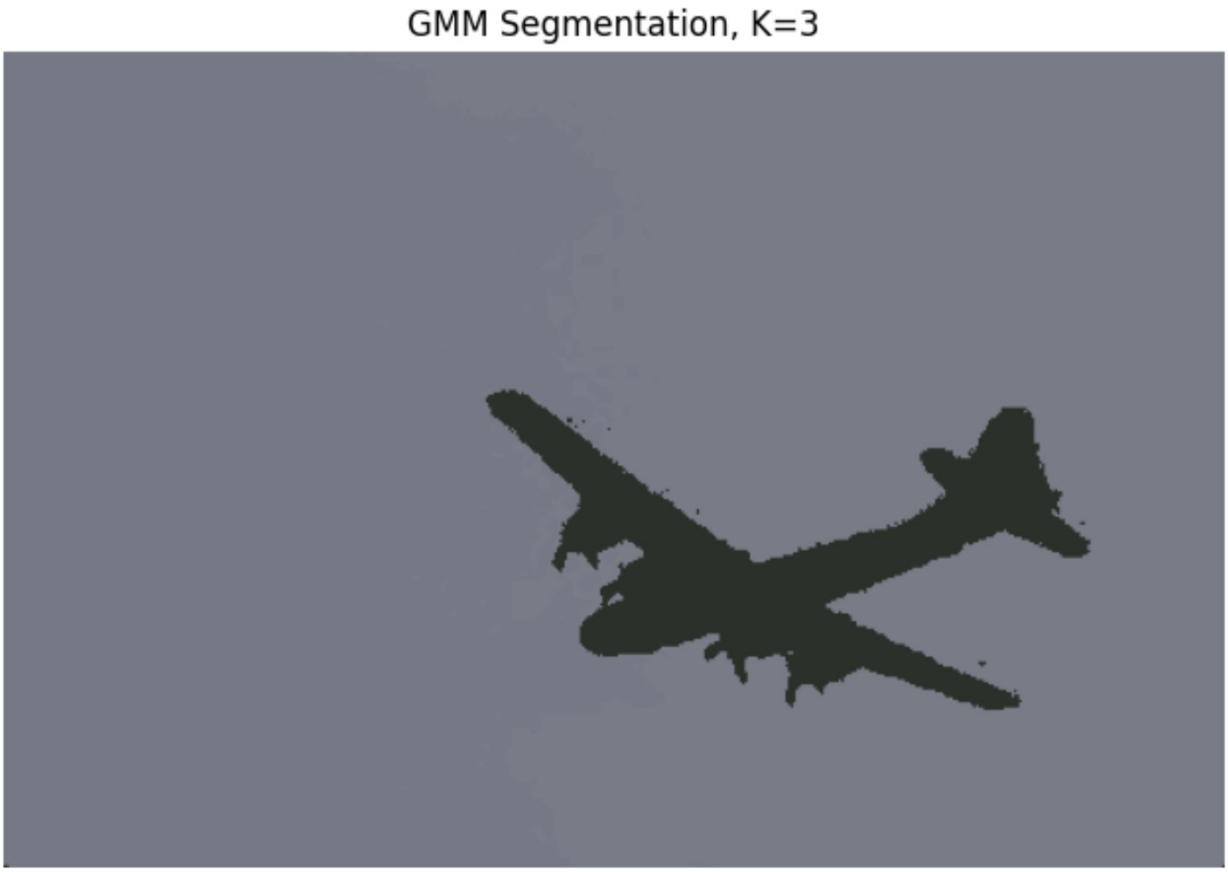


Figure 11: GMM with K = 3



Figure 12: K-Means with K = 4

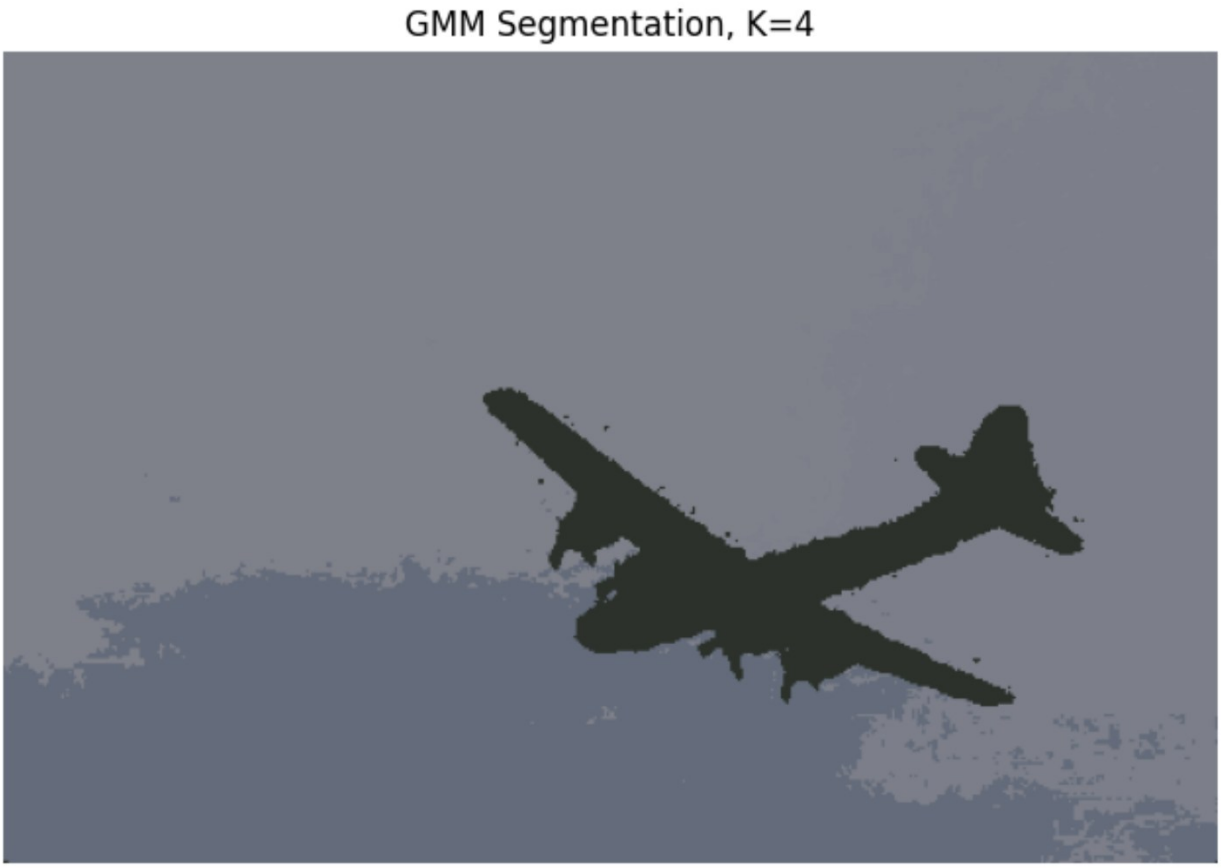


Figure 13: GMM with K = 4

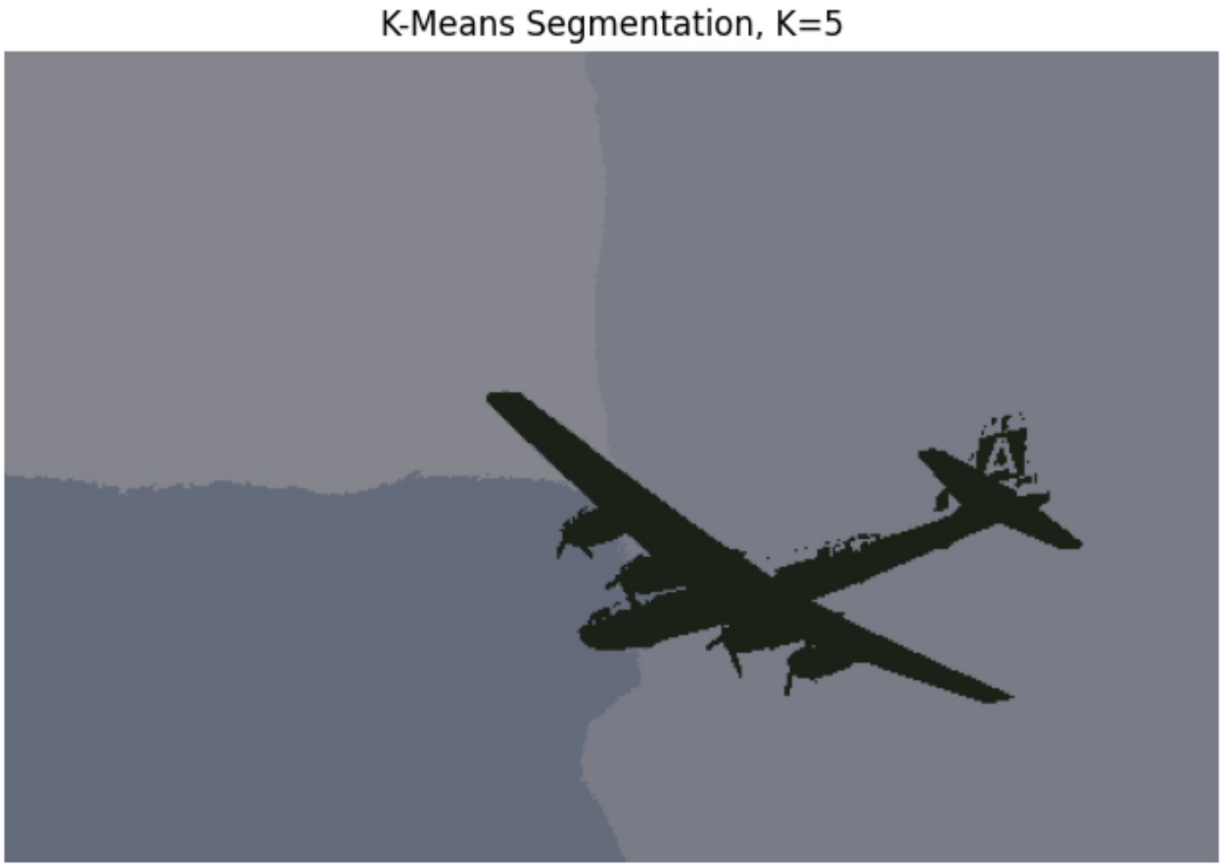


Figure 14: K-Means with K = 5

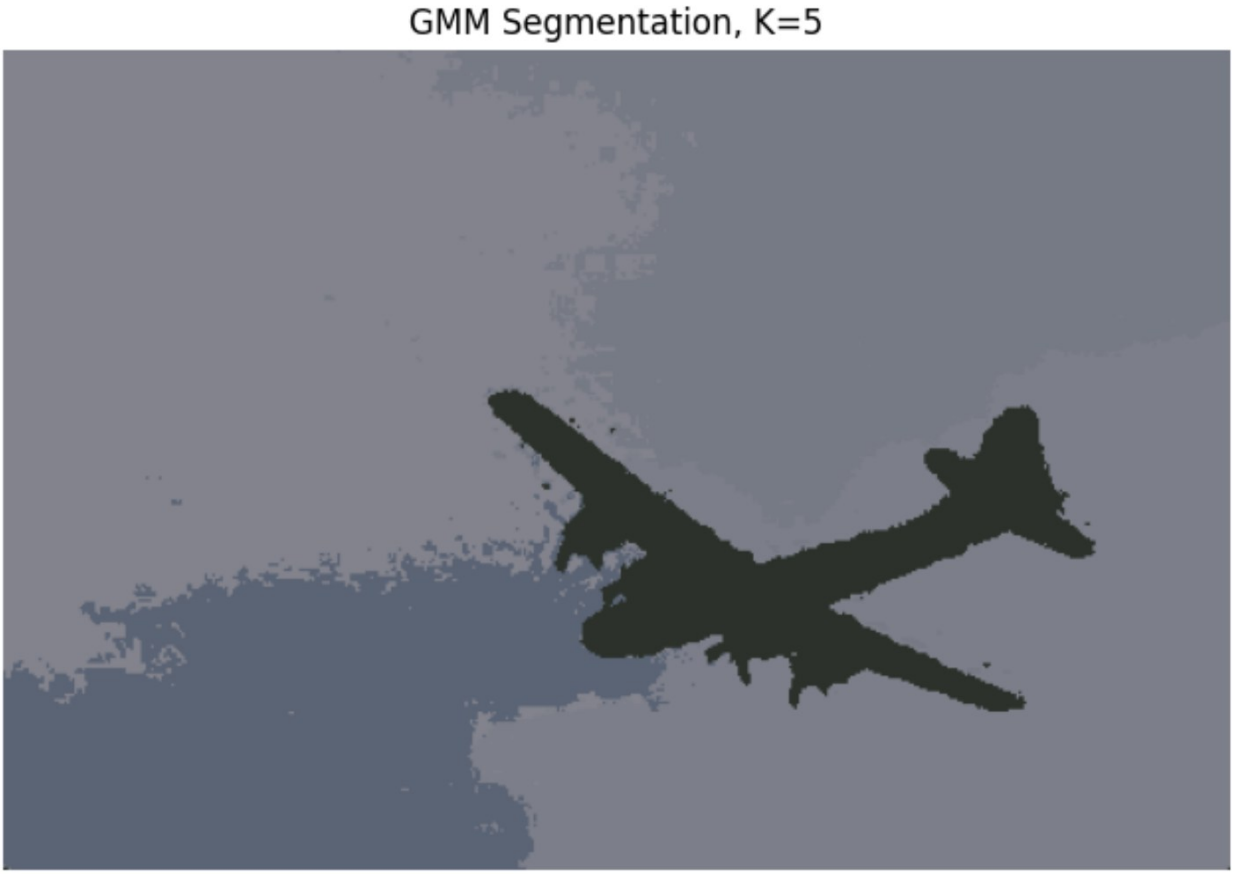


Figure 15: GMM with K = 5

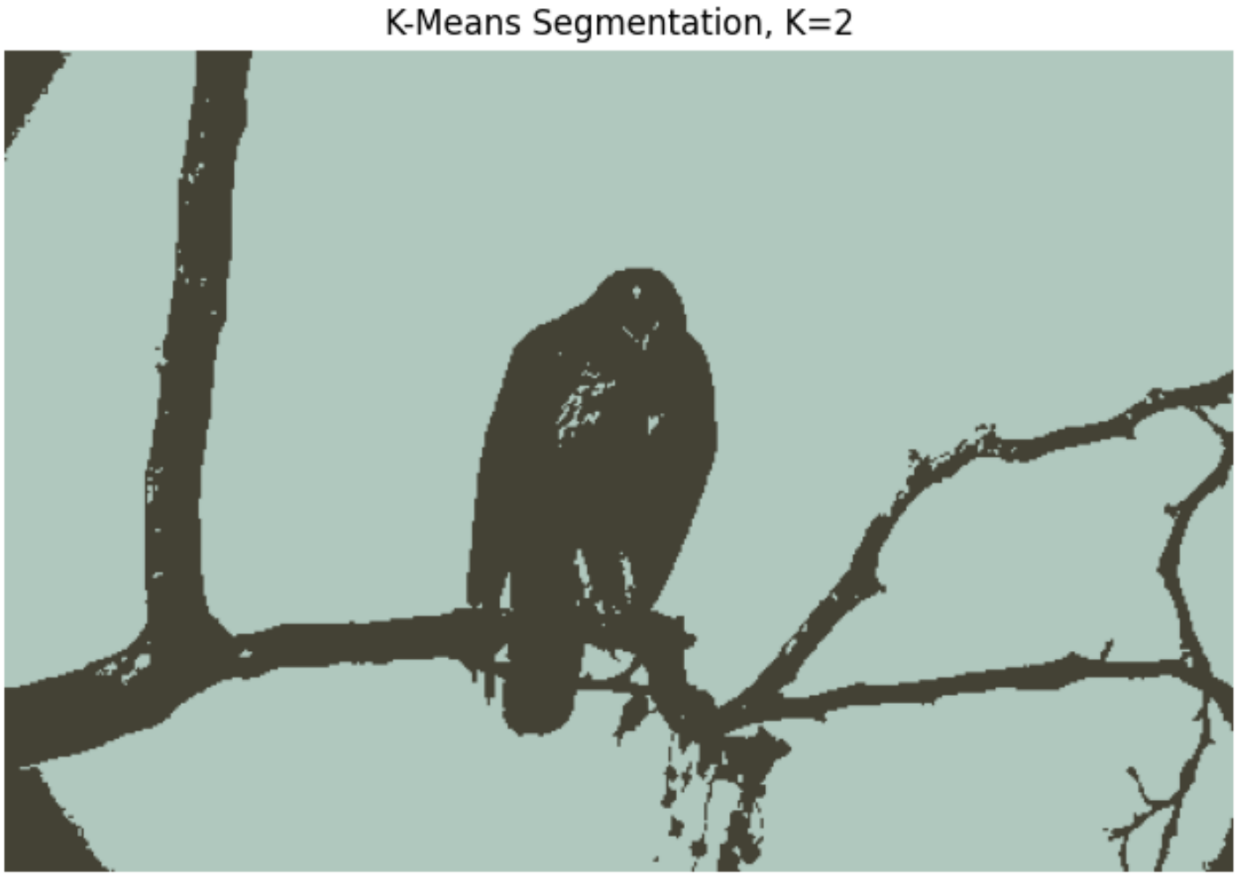


Figure 16: K-Means with K = 2



Figure 17: GMM with K = 2

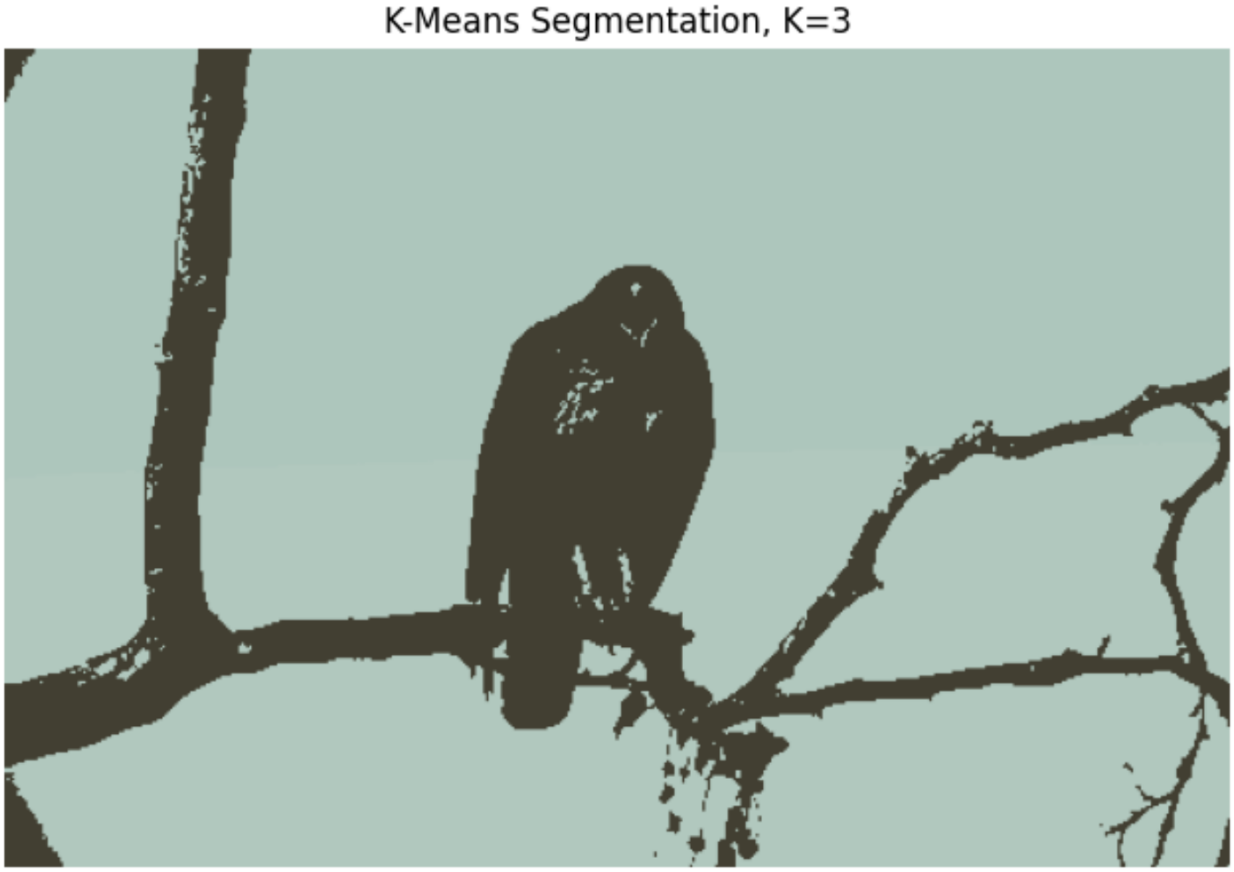


Figure 18: K-Means with K = 3



Figure 19: GMM with K = 3

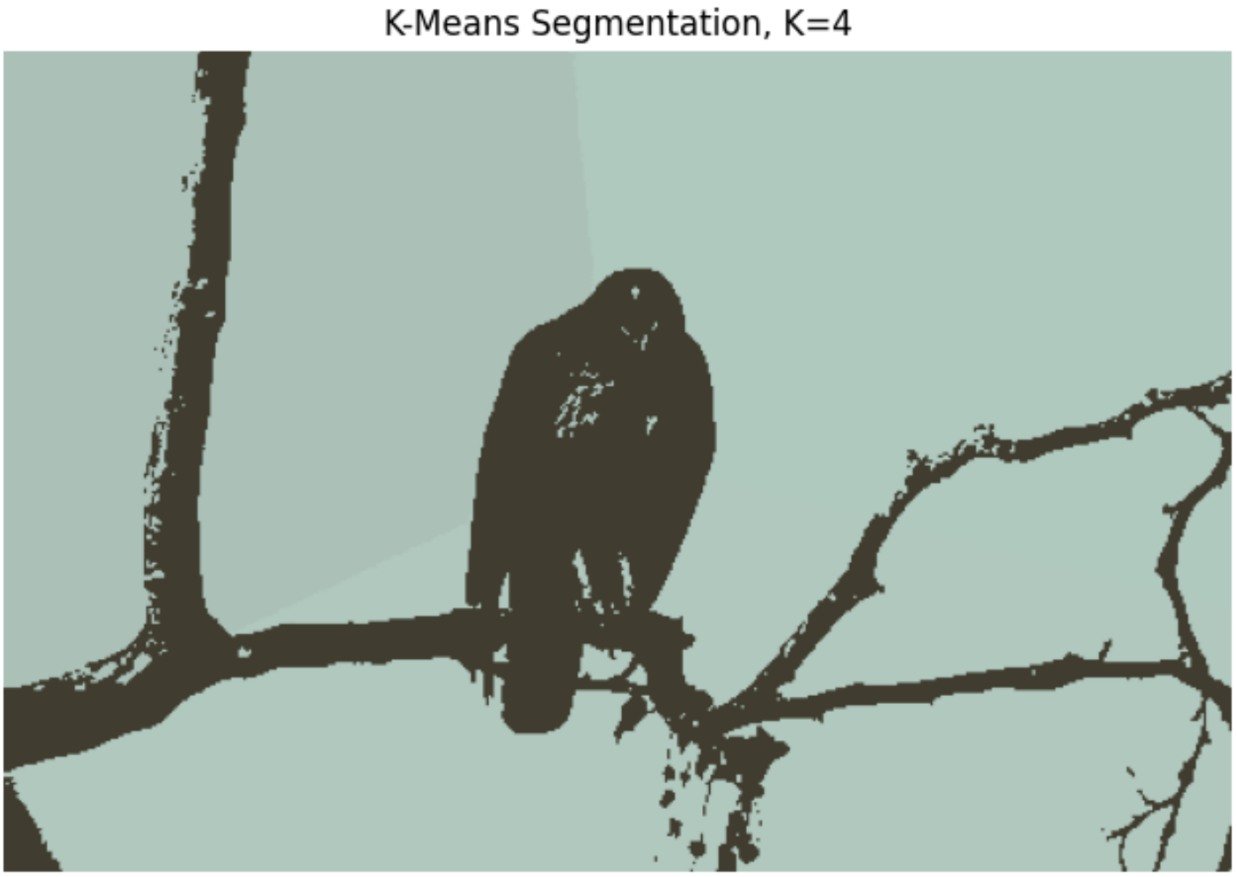


Figure 20: K-Means with K = 4

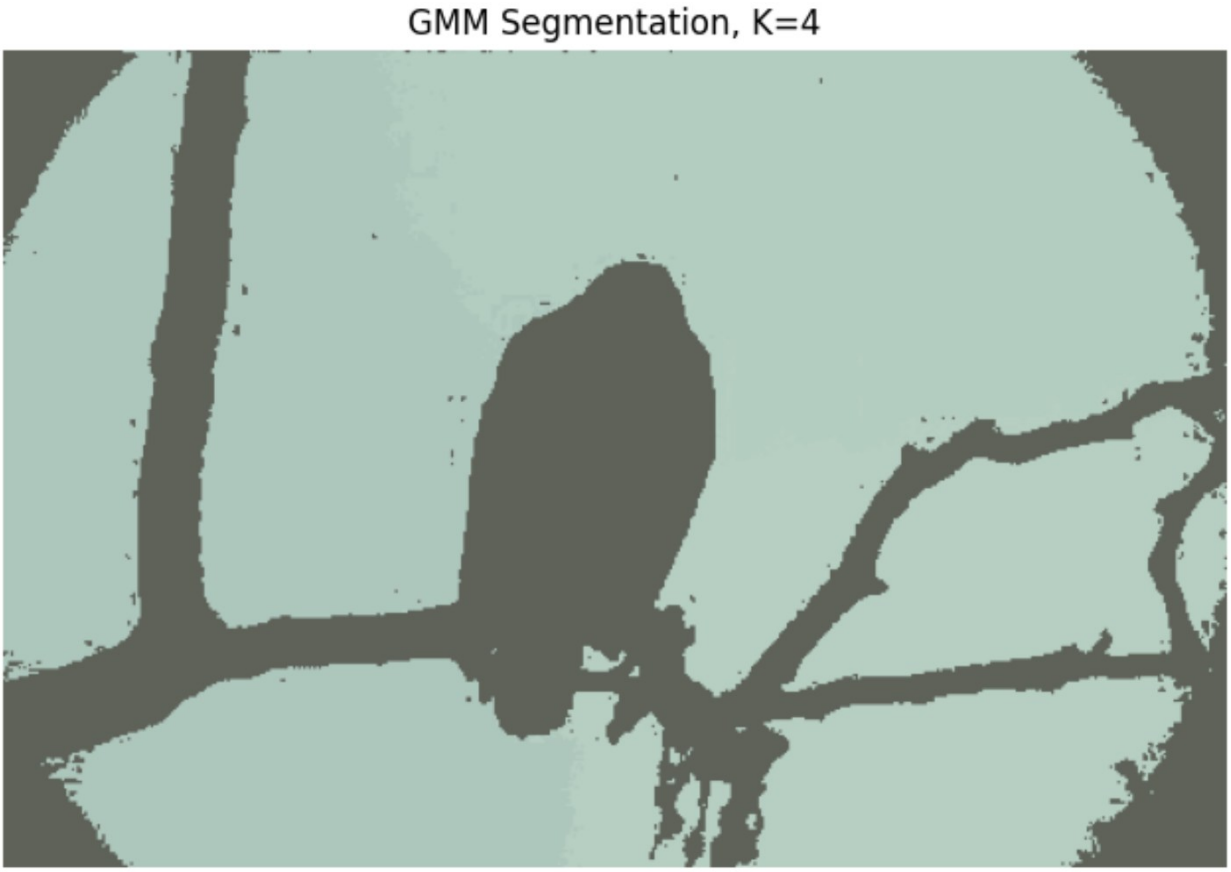


Figure21 : GMM with K = 4

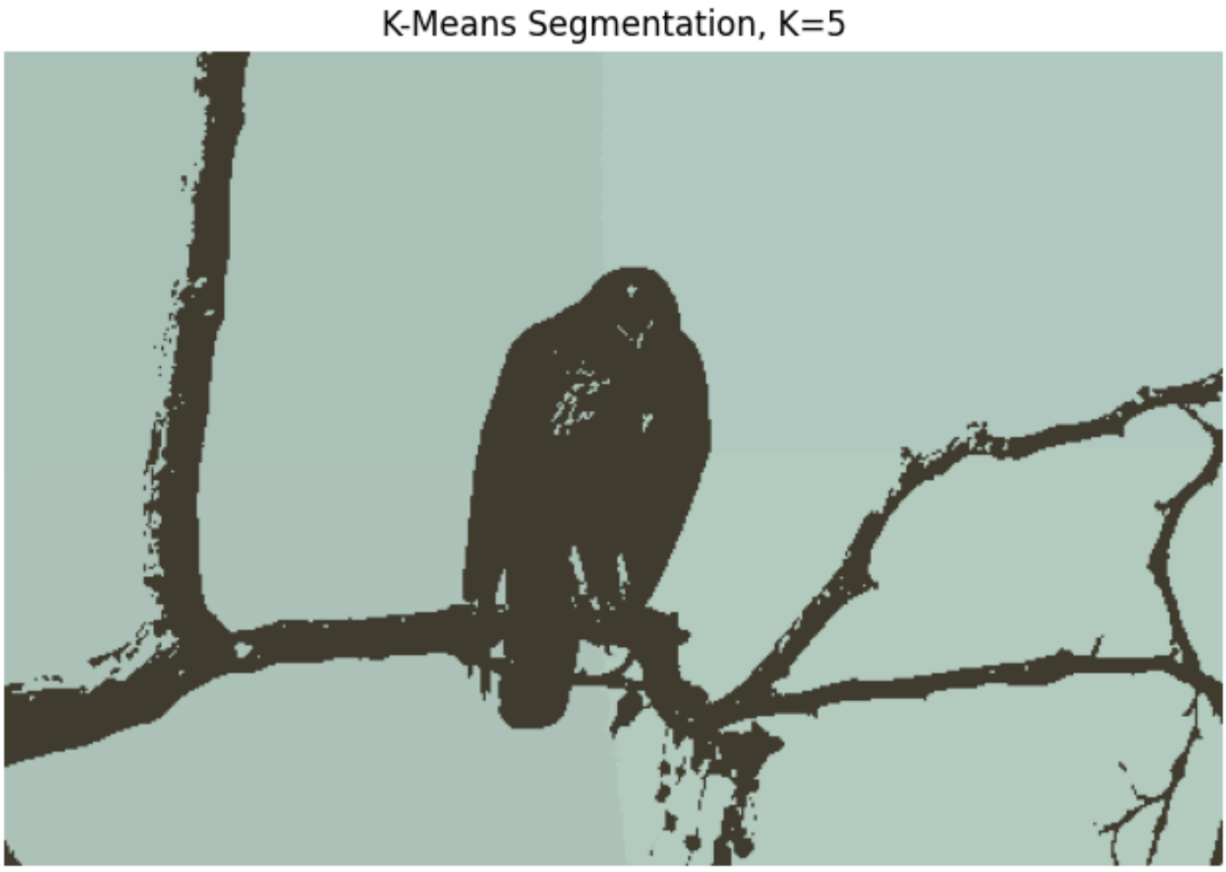


Figure 22: K-Means with K = 5



: GMM K = 523 Figure

6. Conclusion

- Both K-Means and GMM effectively segment images using spatial + color features.
- **K-Means** is simpler and faster but may produce less natural boundaries.
- **GMM** can better handle irregular and overlapping clusters, producing smoother segmentation.
- Using **5D normalized features** (row, column, RGB) ensures spatial coherence and color consistency.
- Choice of KKK is application-dependent: small K for coarse segmentation, larger K for finer details.

This clustering-based approach is a **foundation for advanced image segmentation tasks** such as object detection or pre-processing for computer vision models.